A
PROJECT REPORT
ON

MEDIPHARMA HUB

YOUR TRUSTED STANDARD FOR
AUTHENTIC & PRESCRIBED MEDICINES

(ONLINE PHARMA & MEDICINE ORDER PORTAL)

SUBMITTED BY

RAJVI SURESHBHAI VASOYA

ACADEMIC YEAR 2026-27
T.Y B.C.A SEM -5

UNDER THE GUIDANCE OF

Mr. SHAUNAK PUROHIT

SMT. C.Z.M GOSRANI B.C.A. COLLEGE
JAMNAGAR

SUBMITTED TO

SAURASHTRA UNIVERSITY
RAJKOT

A
PROJECT REPORT
ON

MEDIPHARMA HUB

YOUR TRUSTED STANDARD FOR
AUTHENTIC & PRESCRIBED MEDICINES

(ONLINE PHARMA & MEDICINE ORDER PORTAL)

SUBMITTED BY

RAJVI SURESHBHAI VASOYA

ACADEMIC YEAR 2026-27
T.Y B.C.A SEM -5

UNDER THE GUIDANCE OF

Mr. SHAUNAK PUROHIT

SMT. C.Z.M GOSRANI B.C.A. COLLEGE
JAMNAGAR

SUBMITTED TO

SAURASHTRA UNIVERSITY
RAJKOT

OSHWAL EDUCATION TRUST MANAGED
Smt. Chandramaniben Zaverchand Meghji Gosrani BCA College
Shah Bhagvandji Kachrabhai Edu. Complex, Ahead of Oshwal Circle, Indira Gandhi Marg, Jamnagar - 361004, Gujarat, India
Web: czmgbca.oshwaleducationtrust.org Email: info.czmgbca@oshwaleducationtrust.org
Tel: +912882573475 +912882563886

AFFILIATED TO SAURASHTRA UNIVERSITY, RAJKOT & APPROVED BY AICTE

CZMG BCA-135/2026-27/200 Date: 01/10/2026

CERTIFICATE OF PROJECT COMPLETION

This is to certify that project
"Online pharma and medicine order portal"
is a bonafied work carried out by
Vasoya Rajvi S. (24CS002UG04389)
as a partial fulfillment of the requirements
for the B.C.A. Semester 5 during the
Academic Year 2026-2027

The project report meets the academic requirements for the project work prescribed by Saurashtra University, Rajkot.

EXAMINER NAME & SIGN	**PRINCIPAL SIGN**
Mr. Shaunak Purohit	/ /2026
PROJECT GUIDE	**EXAM DATE**

INDEX

Sr. No.	Particulars	Page No.
1	**Certificate**	**3**
2	**Abstract**	**5**
3	**Acknowledgement**	**6**
4	**Project Profile**	**7**
5	**SDLC Overview**	**8**
6	**Requirement Gathering and Analysis**	**9**
	2.1 Organizational Details	9
	2.2 Meetings	9
	2.3 Data Which Will Be Input into System	10
	2.4 Data Which Will Be Output from System	10
	2.5 Type of Project	10
	2.6 Method of Collecting Data	10
7	**System Requirement Specifications**	**11**
	3.1 Introduction	11
	3.2 Overall Description	11
	3.3 External Interface Requirement	13
	3.4 System Features	13
	3.5 Other Nonfunctional Requirements	14
	3.6 Feasibility Study	15
8	**System Analysis and Modeling**	**16**
	4.1 Use Case Diagram	16
	4.2 Normalization and E-R Diagram	18
	4.3 Data Dictionary	22
	4.4 Functional and Behavior Modeling	25
	4.5 Gantt Chart	28
9	**Test Case**	**29**
10	**Screenshots**	**41**
	User Side	41
	Admin Side	53
11	**Code Explanation**	**60**
	7.1 Common Files	58
	7.2 Admin Side Files	62
	7.3 User Side Files	67
	7.4 Database	73
12	**Limitations and Future Enhancements**	**75**
	8.1 Limitations	75
	8.2 Future Enhancements	75
13	**Conclusion**	**77**
14	**References and Bibliography**	**78**

ABSTRACT

MEDIPHARMA HUB is a comprehensive web-based platform designed to bridge the gap between patients and pharmacies by digitizing the medication procurement process. In an era where convenience and accessibility are paramount, this project aims to provide a secure, user-friendly, and efficient ecosystem for managing medicine orders and inventory.

The core objective of this system is to facilitate a seamless user experience, allowing customers to search for medicines across various categories, securely upload physician prescriptions, and place orders directly through the portal.

From an administrative perspective, the platform manages medicine stock and alerts the administrator when the stock reaches the minimum threshold. This feature automatically alerts the administrator whenever product quantities fall below a predefined threshold, effectively preventing stock-outs and ensuring business continuity. By automating these critical workflows, the project significantly enhances operational efficiency and minimizes human error in the pharmacy supply chain.

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my project guide, Mr. SHAUNAK PUROHIT, for his invaluable mentorship, patience, and encouragement throughout the development of this project. His expert insights and constructive feedback were instrumental in refining the technical architecture and overall functionality of the MEDIPHARMA HUB.

I am also deeply grateful to our Principal ma'am, Mrs. Hetal Savla, for providing an excellent academic environment and the necessary infrastructure that made the completion of this project possible. I further extend my thanks to all the faculty members of the Smt. Chandramaniben Zaverchand Meghji Gosrani BCA College providing the academic environment and resources necessary to successfully complete this endeavor. Their guidance has been a constant source of inspiration and growth.

Finally, I would like to extend my heartfelt thanks to my family and friends for their unwavering support and motivation during the course of this project. Their belief in my efforts pushed me to overcome challenges and strive for excellence in every phase of this development.

PROJECT PROFILE

DEVELOPER	
Name: **RAJVI S. VASOYA**	**Enrollment Number:** **24CS002UG04389**
PROJECT DETAILS	
Project Title:	**Online pharma and medicine order portal**
Duration:	**3 Months**
Name of Project:	**MEDIPHARMA HUB**
Platform:	**HTML, CSS, JS (Front End)** **PHP, MySQL (Back End)**
Team Size:	**1**
GUIDE INFORMATION	
Name of Guide:	
Mr. Shaunak Purohit	

1. SDLC OVERVIEW



2. REQUIREMENT GATHERING AND ANALYSIS

2.1 ORGAIZATIONAL DETAILS:

i. NAME OF ORGANIZATION:

LifeCare Medical & Pharmacy

ii. BRIEF DETAIL OF ORGANIZATION:

LifeCare Medical & Pharmacy is a retail pharmacy that provides medicines and healthcare products. It offers genuine medicines, quality customer service and home delivery. The organization keeps its medicine stock, customer records and orders in a pharmacy management system.

2.2 MEETINGS:

i. MEETING WITH PROJECT GUIDE:

a) NAME OF GUIDE:

Mr. Shaunak Purohit

b) GUIDE'S REQUIREMENT IN HIS WORDS:

A system is required to manage online medicine ordering, prescription handling, medicine inventory and customer management efficiently.

ii. MEETING WITH USER:

a) NAME OF THE USER:

Customer / Patient

b) USER'S REQUIREMENT IN THEIR WORDS:

A convenient online platform is required where users can search medicines, upload a prescription when it is needed, place orders and track or cancel them.

2.3 DATA WHICH WILL BE INPUT INTO SYSTEM:

- Customer Registration Details
- Login Credentials
- Medicine Search and Filter Details
- Shopping Cart Details
- Doctor Prescription Images/PDF
- Order and Cancellation Details
- Customer Profile Details
- Medicine and Category Details
- Stock and Minimum Threshold Details
- Order Status Details
- Customer Management Details

2.4 DATA WHICH WILL BE OUTPUT FROM SYSTEM:

- Medicine List and Details
- Shopping Cart Summary
- Order Confirmation and History
- Customer Profile Details
- Admin Dashboard Statistics
- Low-Stock Alerts and Details
- Order List with Prescription Links
- Customer List with Order Count and Total Spent

2.5 TYPE OF PROJECT:

- Online Web-Based Application

2.6 METHOD OF COLLECTING DATA:

- Interviews
- Observation
- Requirement Analysis

3. SYSTEM REQUIREMENT SPECIFICATIONS

3.1 INTRODUCTION:

3.1.1 PURPOSE:

MediPharma Hub is an online pharmacy and medicine order portal. Customers can search medicines, upload a prescription when required, order medicines online (cash on delivery) and track or cancel their orders. The pharmacy administrator manages medicines, categories, stock, orders and customers through an admin panel.

3.1.2 PRODUCT SCOPE:

The system is a web application used through a browser on a local XAMPP server. It covers customer registration and login, medicine browsing with search, filters, sorting and pagination, a session-based shopping cart, prescription upload, checkout, order history with cancellation, profile editing and a complete admin panel. Online payment, e-mail / SMS notification and invoice generation are not part of the current system

3.2 OVERALL DESCRIPTION:

3.2.1 PRODUCT FUNCTION:

i. ADMIN MODULE:

- Admin Login and Logout
- Dashboard Statistics
- Low-Stock Alerts
- Medicine and Category Management
- Order Management and Prescription Viewing
- Order Status Updates
- Customer Management and Deletion
- Customer Order History and Total Spent

ii. USER MODULE:

- User Registration and Authentication
- Medicine Browsing, Search, Filters and Pagination
- Shopping Cart Management

- Automatic Cart Total Calculation
- Prescription Upload for Required Medicines
- Checkout and Cash on Delivery
- My Orders and Order Cancellation
- View and Edit Customer Profile

3.2.2 USER CLASSES AND CHARACTERSTICS:

i. USER CLASSES:

- **Administrator:** manages medicines, categories, orders and customers and has higher privileges than a customer. Only the account with role "admin" can open the pages inside the admin folder.
- **Customer:** registers through the registration form (role is always "customer"), browses medicines and can use the cart, checkout, My Orders and Profile pages only after login.

ii. USER CHARACTERSTICS:

- **Administrator:** should know how to manage medicine records, stock levels and orders through the admin panel.
- **Customer:** needs only basic knowledge of web browsing and online shopping.

3.2.3 OPERATING ENVIRONMENT:

The system runs on a computer with XAMPP (Apache, MySQL, phpMyAdmin) and is accessed through a web browser on the local server. Windows is used as the development environment.

3.2.4 DESIGN AND IMPLEMENTATION CONSTRAINTS:

- The graphical user interface is available in English only.
- An internet connection is needed.

- Prescription upload accepts JPG, JPEG, PNG and PDF files up to 5 MB; medicine photos accept JPG, PNG and WEBP up to 2 MB.
- The database connection settings (host, port, user, password) are written in config/db.php and must match the local MySQL server.

3.2.5 ASSUMPTIONS AND DEPENDENCIES:

- The administrator account is created from database.sql before the system is used.
- Medicine data is added and updated by the administrator only.
- User roles are predefined: "customer" and "admin".
- Payment is made as Cash on Delivery when the order is delivered.

3.3 EXTERNAL INTERFACE REQUIREMENT:

3.3.1 USER INTERFACE:

- Customers register, log in, search and view medicines, add them to the cart, upload a prescription, place orders, view / cancel orders and edit their profile.
- The system shows an error message for incorrect login details or invalid input.
- The administrator logs in and manages medicines, categories, orders, customers and stock from the admin panel.

3.3.2 SOFTWAE INTERFACE:

- Operating System – Windows
- Front End – HTML, CSS, JavaScript
- Back End – PHP
- Database – MySQL
- Server – Apache Server
- Development Environment – Visual Studio Code
- Communication Interface – Web Browser

3.4 SYSTEM FEATURES:

- A customer can register with name, email, phone, address and a confirmed password; the email must be unique.

- Customers and the administrator can log in and log out; the administrator is sent to the dashboard, a customer to the medicine list.
- The medicine list can be searched by name, filtered by category, price range and availability, sorted and paged.
- A logged-in customer can add medicines to the cart, change quantities within the available stock and remove items.
- Prescription upload is required before ordering a prescription-only medicine and is not shown otherwise.
- A customer can place an order (Cash on Delivery); stock is reduced and the cart is cleared.
- A customer can view own orders and cancel an order that is still pending or processing; stock is restored.
- A customer can view and edit own profile details.
- The administrator can add, edit and delete medicines and categories.
- The administrator sees a low-stock alert for medicines at or below their minimum threshold.
- The administrator can view all orders, open prescription files and change the order status.
- The administrator can view customers with order count and total spent and delete a customer account.

3.5 OTHER NONFUNCTIONAL REQUIREMENTS:

3.5.1 PERFORMANCE REQUIREMENT:

- Pages should respond quickly for normal use; searching, filtering and paging are done in the database.
- Cart quantity changes update the page without a full reload when JavaScript is available.
- Database errors are reported as friendly messages and do not stop the whole site.

3.5.2 SECURITY REQUIREMENT:

- Passwords are stored as hashes (password_hash) and checked with password_verify.
- All database queries use PDO prepared statements; output is escaped with htmlspecialchars.
- Admin pages are protected by a role check, customer pages by a login check.

- Forms that change data in customer pages carry a CSRF token that the server verifies.
- Uploaded prescription files get a server-generated name and are checked for type, size and real file content.

3.5.3 SAFETY REQUIREMENT:

- Orders are saved inside a database transaction so that an order and its stock change are stored together or not at all.
- Regular backup of the medipharma_hub database should be taken to prevent data loss.

3.6 FEASIBILITY STUDY:

3.6.1 OPERATIONAL FEASIBILITY:

MediPharma Hub gives customers an easy way to order medicines online and helps the administrator manage stock and orders with less manual work.

3.6.2 TECHNICAL FEASIBILITY:

The system uses widely available technologies (HTML, CSS, JavaScript, PHP, MySQL) and runs on an ordinary computer with XAMPP, so it is technically feasible.

3.6.3 ECONOMICA FEASIBILITY:

All tools used (XAMPP, PHP, MySQL, Visual Studio Code) are free and open source, so the development cost is very low while paperwork and manual effort are reduced.

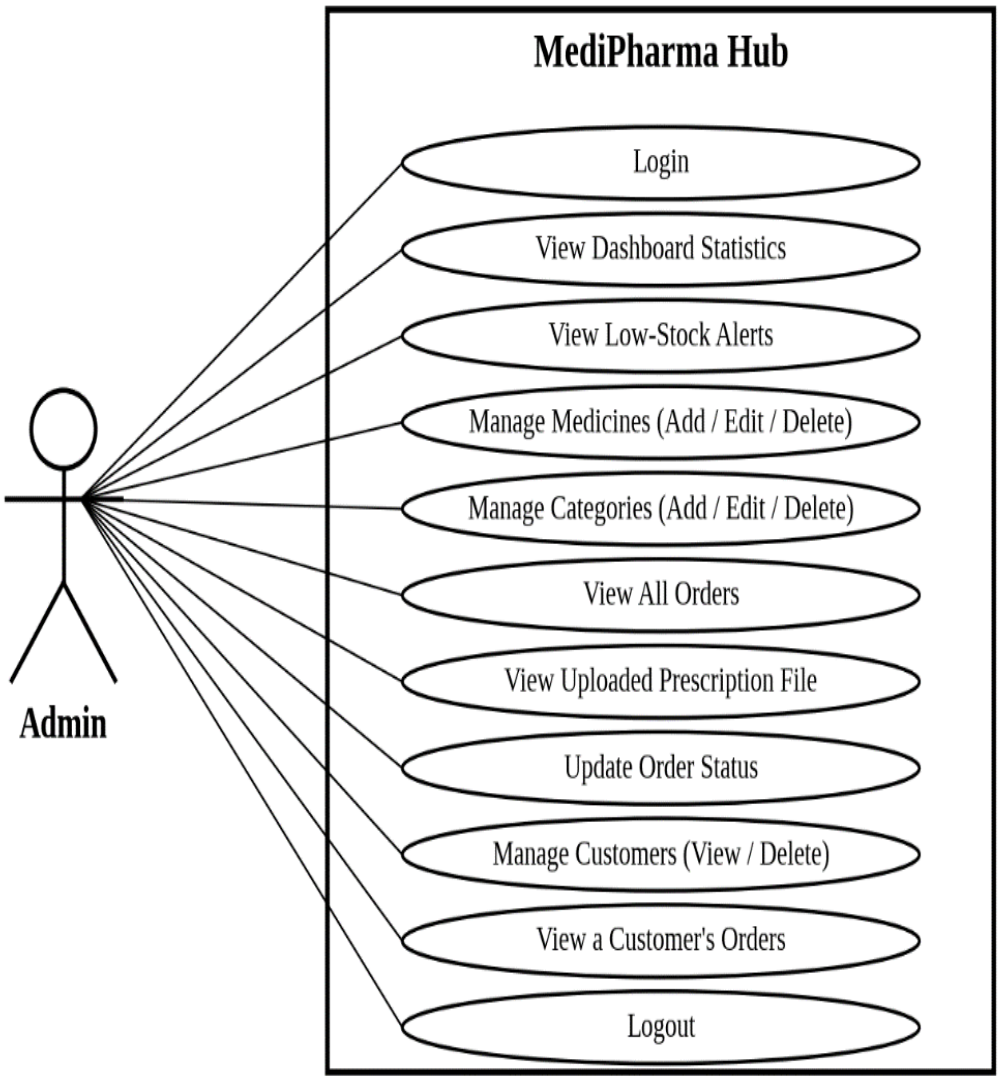
3.6.4 SCHEDULE FEASIBILITY:

`The project was planned for three months with familiar technologies; the Gantt chart in Chapter 4 shows the planned and actual time of every task.

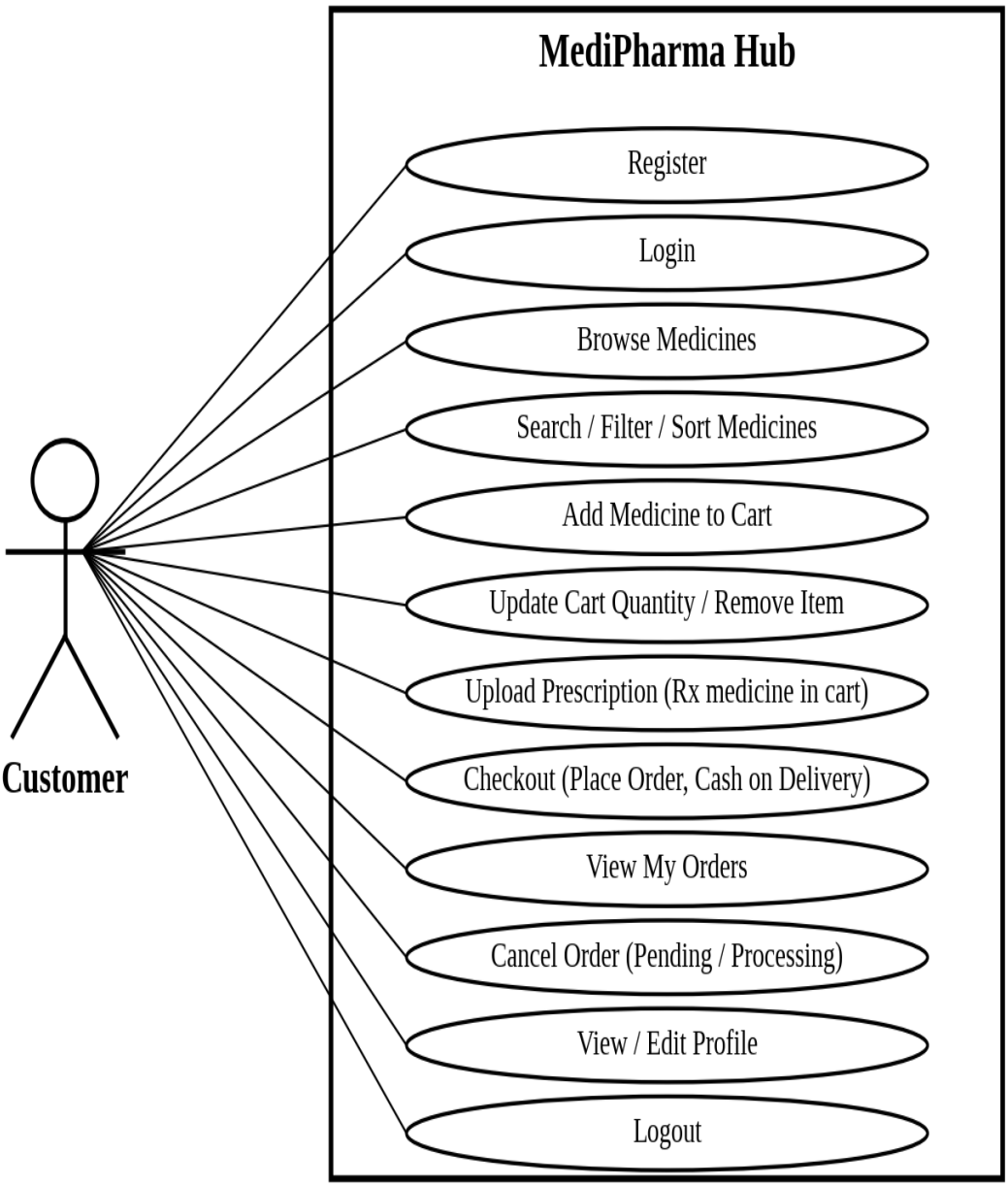
4. SYSTEM ANALYSIS AND MODELING

4.1 USE CASE DIAGRAM:

i. ADMIN SIDE:



ii. USER SIDE:


MediPharma Hub
Register
Login
Browse Medicines
Search / Filter / Sort Medicines
Add Medicine to Cart
Update Cart Quantity / Remove Item
Upload Prescription (Rx medicine in cart)
Checkout (Place Order, Cash on Delivery)
View My Orders
Cancel Order (Pending / Processing)
View / Edit Profile
Logout
Customer

4.2 NORMALIZATION AND E-R DIAGRAM:

Normalization is the process of organizing the database into separate and well-structured tables (relations) and the columns (attributes) to reduce data redundancy and improve data integrity.

→ DE-NORMALIZED FORM:

order_ id	user_ id	full_ name	email	phone	address
1	2	Asha Patel	asha@example.com	9876543210	12, Sample Road, Jamnagar
2	2	Asha Patel	asha@example.com	9876543210	12, Sample Road, Jamnagar

prescription_ path	total_ amount	status	created_ at	medicine_ id
assets/uploads/rx_2_….jpg	300.00	pending	2026-10-05 11:20	31 1
NULL	300.00	pending	2026-10-07 16:45	1

medicine_ name	category_ id	category_ name	requires_ prescription	quantity	price
Metformin Tablets Vitamin C Tablets	3 1	Diabetes Care Nutritional Supplements	1 0	2 1	100.00 100.00
Vitamin C Tablets	1	Nutritional Supplements	0	3	100.00

→ NORMAILZED FORM:

(A) 1st Normal Form(1NF):

order_id	user_id	full_name	email	medicine_id	medicine_name	category_id	category_name	quantity	price
101	5	Asha Patel	asha@example.com	21	Metformin Tablets	3	Diabetes Care	2	100.00
101	5	Asha Patel	asha@example.com	1	Vitamin C Tablets	1	Nutritional Supplements	1	100.00
102	5	Asha Patel	asha@example.com	1	Vitamin C Tablets	1	Nutritional Supplements	3	100.00

(B) 2nd Normal Form(2NF):

2NF requires that every non-key attribute depends on the whole key and not on a part of it. In the 1NF relation, order_id alone determines the order and customer attributes, and medicine_id alone determines the medicine and category attributes. Only quantity (and the price paid) depend on the complete key (order_id, medicine_id).

The relation is therefore decomposed:

2NF relation	Attributes	Key
ORDER	order_id, order_date, status, prescription_path, order_total, user_id, full_name, email, phone, address	order_id
MEDICINE	medicine_id, medicine_name, category_id, category_name, description, price, stock, min_threshold, requires_prescription, image	medicine_id
ORDER_ITEM	order_id, medicine_id, quantity, price_at_order	(order_id, medicine_id)

(C) 3rd Normal Form(3NF):

3NF requires that no non-key attribute depends on another non-key attribute (no transitive dependency). Two transitive dependencies remain in the 2NF relations:

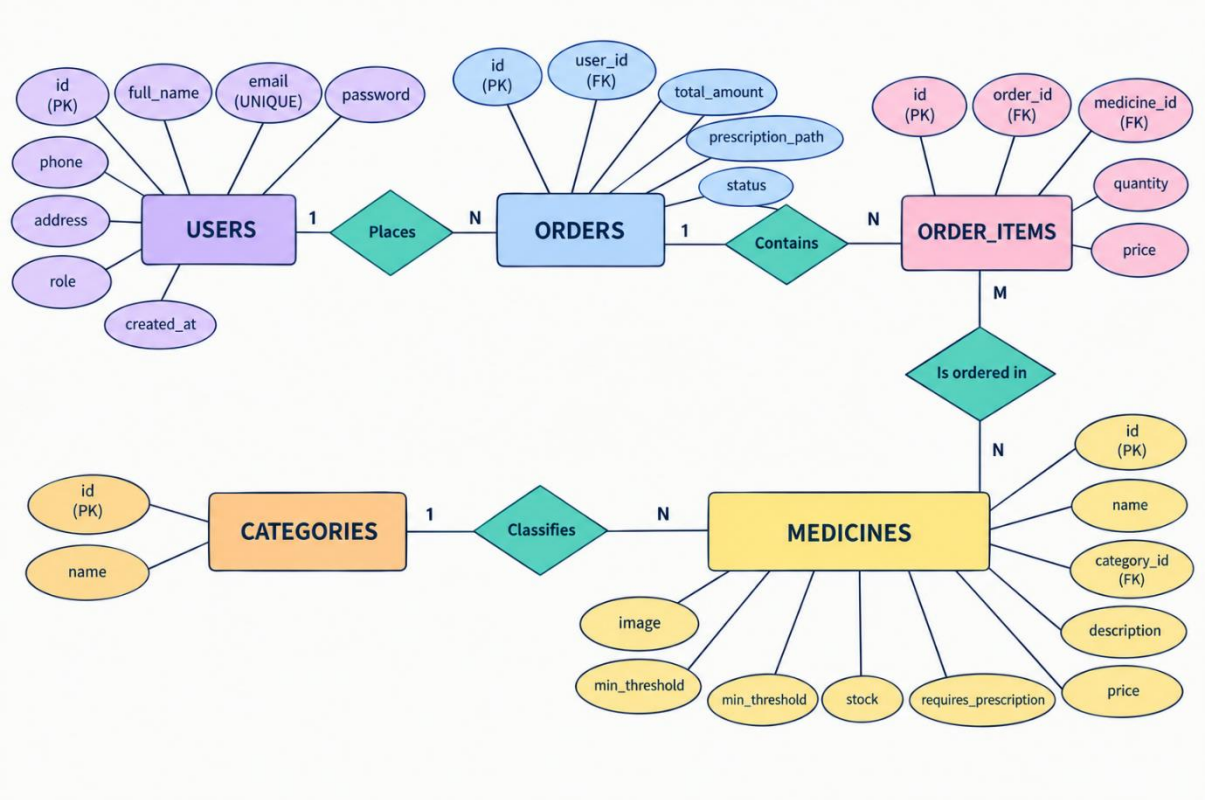
- In ORDER: order_id → user_id → full_name, email, phone, address (customer details depend on the customer, not on the order).

- In MEDICINE: medicine_id → category_id → category_name (the category name depends on the category, not on the medicine).

Moving these attributes into their own relations gives the five tables of MediPharma Hub:

3NF relation (table)	Attributes	Primary key	Foreign key(s)
users	id, full_name, email, password, phone, address, role, created_at	id	–
categories	id, name	id	–
medicines	id, name, category_id, description, price, stock, min_threshold, requires_prescription, image, created_at	id	category_id → categories.id
orders	id, user_id, total_amount, prescription_path, status, created_at	id	user_id → users.id
order_items	id, order_id, medicine_id, quantity, price	id	order_id → orders.id medicine_id → medicines.id

→E-R DIAGRAM:


id (PK)
full_name
email (UNIQUE)
password
phone
address
role
created_at
USERS
1
Places
N
id (PK)
user_id (FK)
total_amount
prescription_path
status
ORDERS
1
Contains
N
id (PK)
order_id (FK)
medicine_id (FK)
quantity
price
ORDER_ITEMS
M
Is ordered in
N
id (PK)
name
category_id (FK)
description
price
requires_prescription
stock
min_threshold
min_threshold
image
MEDICINES
N
Classifies
1
CATEGORIES
id (PK)
name

4.3 DATA DICTIONARY:

- **users**

Column	Data type	Constraint	Description
id	INT	PRIMARY KEY, AUTO_INCREMENT	Unique user number
full_name	VARCHAR(100)	NOT NULL	Name of the user
email	VARCHAR(150)	NOT NULL, UNIQUE	Login email
password	VARCHAR(255)	NOT NULL	Password hash (password_hash)
phone	VARCHAR(20)	NULL allowed	Phone number
address	VARCHAR(255)	NULL allowed	Delivery address
role	ENUM('customer','admin')	NOT NULL, DEFAULT 'customer'	Type of user
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Registration time

- **categories**

Column	Data type	Constraint	Description
id	INT	PRIMARY KEY, AUTO_INCREMENT	Unique category number
name	VARCHAR(100)	NOT NULL	Category name

- **medicines**

Column	Data type	Constraint	Description
id	INT	PRIMARY KEY, AUTO_INCREMENT	Unique medicine number
name	VARCHAR(150)	NOT NULL	Medicine / product name
category_id	INT	FOREIGN KEY → categories(id), ON DELETE SET NULL	Category of the medicine
description	TEXT	NULL allowed	Short description
price	DECIMAL(10,2)	NOT NULL, DEFAULT 0	Selling price in rupees
stock	INT	NOT NULL, DEFAULT 0	Available quantity
min_threshold	INT	NOT NULL, DEFAULT 10	Low-stock alert level
requires_prescription	TINYINT(1)	NOT NULL, DEFAULT 0	1 = prescription needed
image	VARCHAR(255)	DEFAULT NULL	Photo file name
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Time the record was added

- **orders**

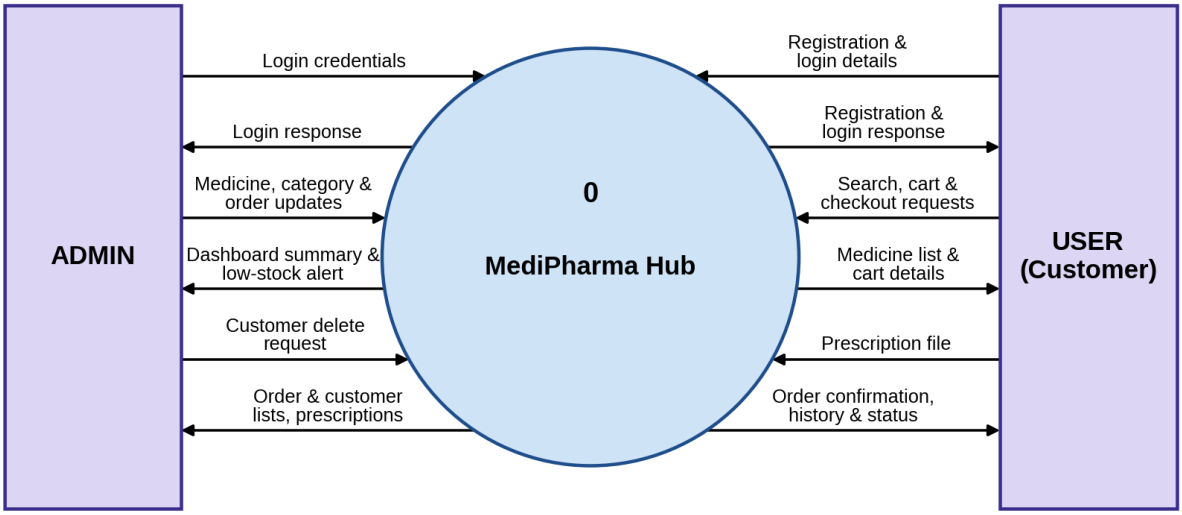
Column	Data type	Constraint	Description
id	INT	PRIMARY KEY, AUTO_INCREMENT	Unique order number
user_id	INT	NOT NULL, FOREIGN KEY → users(id), ON DELETE CASCADE	Customer who placed the order
total_amount	DECIMAL(10,2)	NOT NULL, DEFAULT 0	Order total in rupees
prescription_path	VARCHAR(255)	DEFAULT NULL	Path of the uploaded prescription file
status	ENUM('pending','processing','shipped','delivered','cancelled')	DEFAULT 'pending'	Order status
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Order date and time

- **order_items**

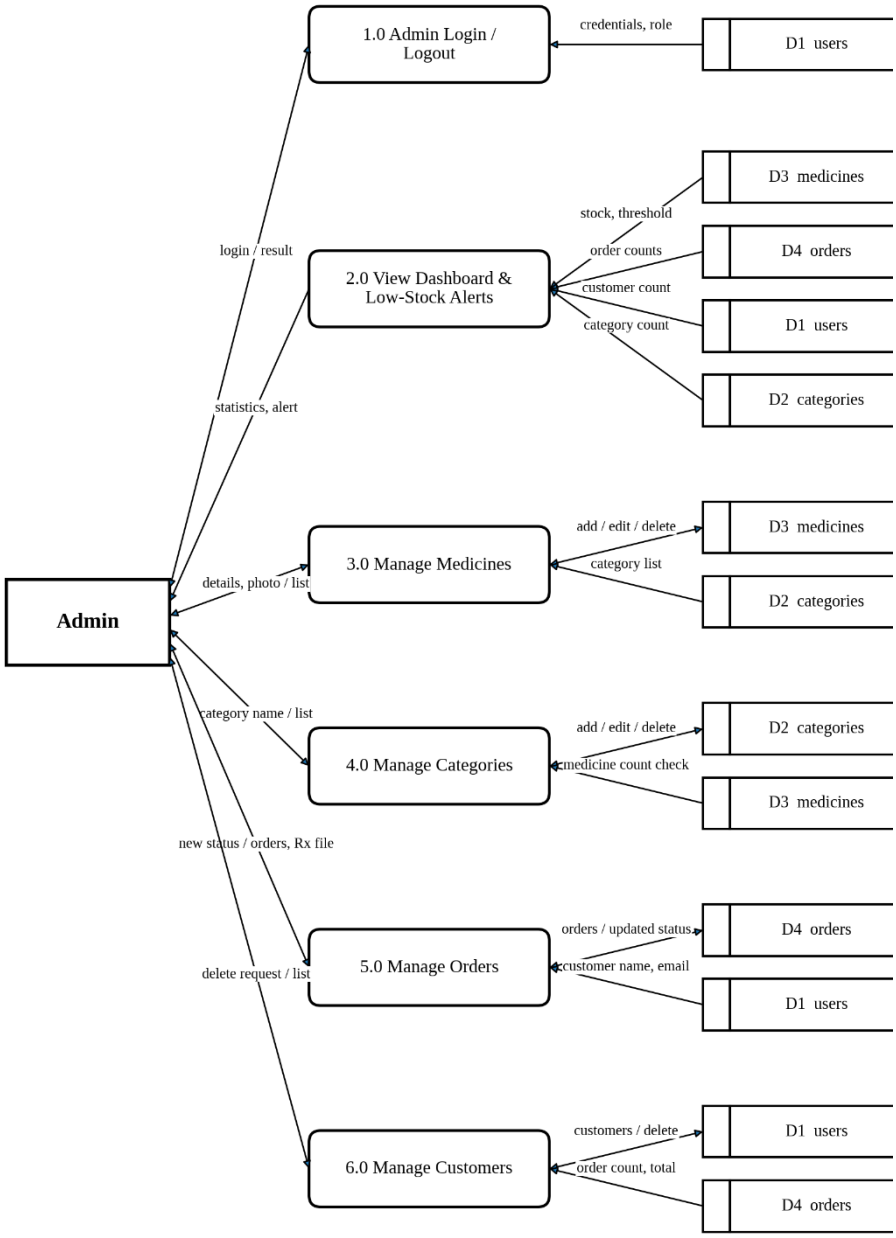
Column	Data type	Constraint	Description
id	INT	PRIMARY KEY, AUTO_INCREMENT	Unique item row number
order_id	INT	NOT NULL, FOREIGN KEY → orders(id), ON DELETE CASCADE	Order the item belongs to
medicine_id	INT	NOT NULL, FOREIGN KEY → medicines(id), ON DELETE CASCADE	Ordered medicine
quantity	INT	NOT NULL	Ordered quantity
price	DECIMAL(10,2)	NOT NULL	Unit price at the time of the order

4.4 FUNCTIONAL AND BEHAVIOR MODELING:

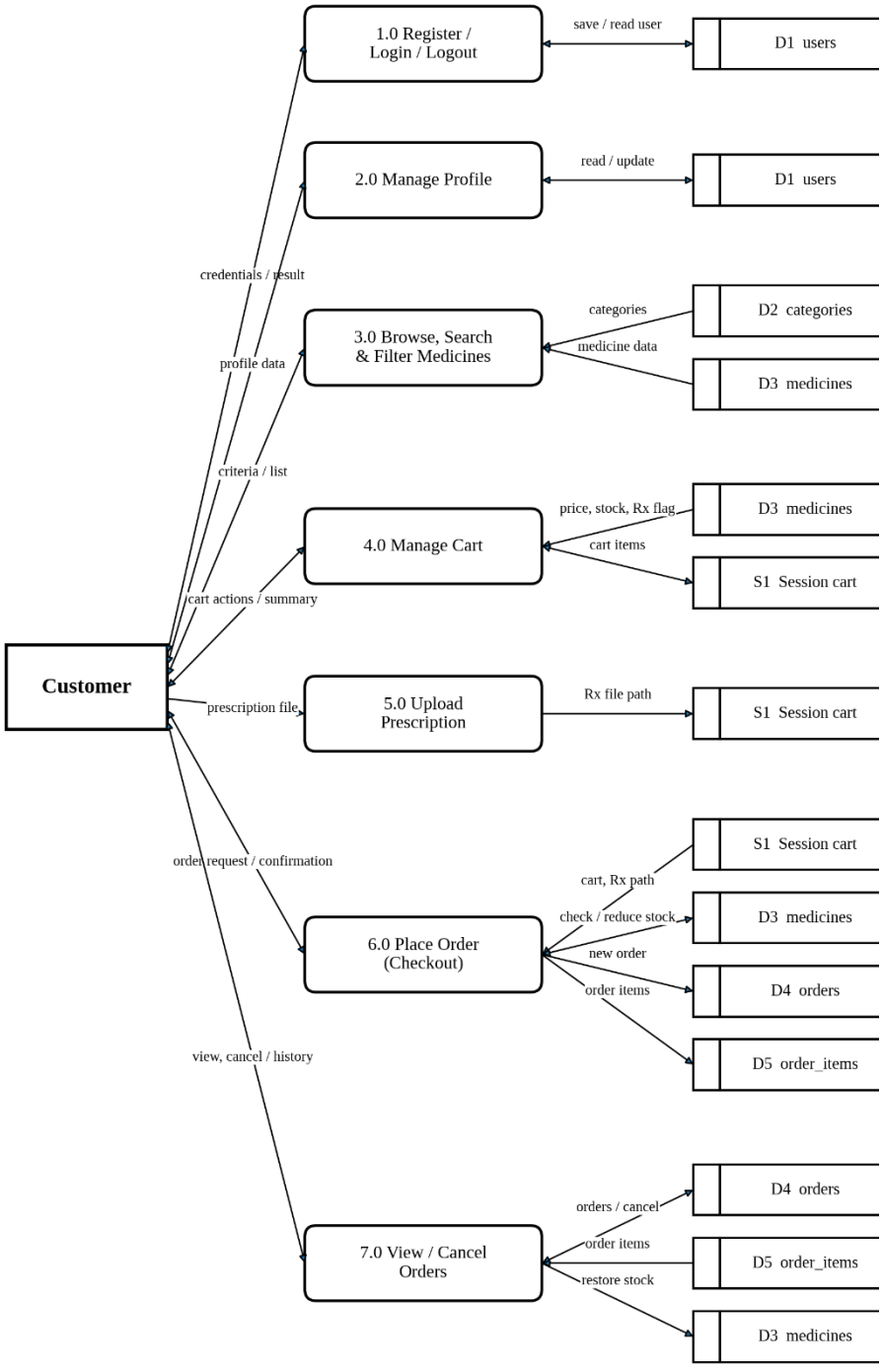
4.4.1 CONTEXT DIAGRAM (0-LEVEL DIAGRAM):



4.4.2 FIRST LEVET DIAGRAM (1-LEVEL DIAGRAM): →ADMIN:



→ USER:


Customer
1.0 Register / Login / Logout
2.0 Manage Profile
3.0 Browse, Search & Filter Medicines
4.0 Manage Cart
5.0 Upload Prescription
6.0 Place Order (Checkout)
7.0 View / Cancel Orders
D1 users
D2 categories
D3 medicines
D4 orders
D5 order_items
S1 Session cart
save / read user
read / update
credentials / result
profile data
categories
medicine data
criteria / list
price, stock, Rx flag
cart items
cart actions / summary
prescription file
Rx file path
order request / confirmation
cart, Rx path
check / reduce stock
new order
order items
view, cancel / history
orders / cancel
order items
restore stock

4.5 GANTT CHART:

→ **Yellow**: Expected Time
→ **Blue**: Actual Time

No.	Task / Time (Week's)	1	2	3	4	5	6	7	8	9	10
1	Project Selection & Planning										
2	System Requirement Analysis										
3	System Design (DFD, ER Diagram, UI Design)										
4	Database Design (Tables, Relationships)										
5	Frontend Development (HTML, CSS)										
6	Backend Development (PHP, MySQL)										
7	Integration & Testing										
8	Debugging & Optimization										
9	Documentation Preparation										
10	Final Review & Submission										

5. TEST CASE

The test cases prepared for the current version of MediPharma Hub. The test cases are written from the actual source code, so the expected results below are what the code is written to do. Each test case has a unique ID and covers one function of the customer side, the admin side or the security checks.

TC-01: User Registration, Login and Access Control Module

Test Case ID	TC-01	**Module**	User Registration, Login and Access Control
Test Title	User Registration, Login and Access Control Test		
Objective	In this test case, the registration, login and logout of the customer and the administrator in MediPharma Hub, the show/hide password button and the protection of the customer and admin pages would be tested.		
Precondition	Registration and Login pages must be working and the administrator account must exist.		
Test Type	Functional / Security	**Priority**	High
Designed On	29/Sept/2026	**Executed On**	05/Oct/2026
Tested By	Rajvi S. Vasoya	**Overall Status**	PASS

Step	Test Step	Test Data	Expected Result	Actual Result	Status
1	Submit Empty Registration Form	All fields empty	Message "Please fill in all required fields." should be displayed	As expected	**PASS**
2	Register New Customer	Rajvi, rajvi@gmail.com, Rajvi@123	New customer account should be created successfully	As expected	**PASS**
3	Login with Wrong Password	rajvi@gmail.com / wrong password	Message "Incorrect email or password." should be displayed	As expected	**PASS**
4	Login with Valid Customer Credentials	rajvi@gmail.com / Rajvi@123	Customer should be logged in and the Medicines page should be displayed	As expected	**PASS**
5	Login with Valid Admin Credentials	admin@medipharma.com / valid password	Admin should be redirected to the Admin Dashboard	As expected	**PASS**

Step	Test Step	Test Data	Expected Result	Actual Result	Status
6	Customer Opens Admin Page	Customer login, Admin Dashboard URL	Customer should be redirected to the Login page	As expected	**PASS**
7	Password and Confirm Password Different	Password: Rajvi@123, Confirm: Rajvi@124	Message "Password and Confirm Password do not match." should be displayed and the form should not be submitted	As expected	**PASS**
8	Register with Existing Email	rajvi@gmail.com again	Message "An account with this email already exists." should be displayed and no new account should be created	As expected	**PASS**
9	Show / Hide Password	Eye button on Login and Register pages	Password field should switch between hidden and visible text and the icon should change	As expected	**PASS**
10	Login with Unknown Email	unknown@gmail.com / any password	Message "Incorrect email or password." should be displayed and no session should be created	As expected	**PASS**
11	Logout	Logout as customer and as admin	Session should be destroyed and the user should be redirected to the Login page	As expected	**PASS**
12	Open Admin Pages without Login	admin/dashboard.php (not logged in)	User should be redirected to the Login page	As expected	**PASS**
13	Open Customer Pages without Login	cart.php, checkout.php, my_orders.php, profile.php, upload_prescription.php	Every page should redirect to the Login page	As expected	**PASS**
14	Admin Opens Customer Pages	Admin login, cart.php and profile.php	Admin should be redirected to the Login page because these pages need the customer role	As expected	**PASS**
15	Back Button after Logout	Logout, Back button, my_orders.php	The page should not be served and the user should be redirected to the Login page	As expected	**PASS**

Step	Test Step	Test Data	Expected Result	Actual Result	Status
16	Request without CSRF Token	POST to add_to_cart.php / update_cart.php without csrf_token	Request should be ignored and the cart should not change	As expected	**PASS**

TC-02: Medicine Search, Filters and Cart Module

Test Case ID	TC-02	**Module**	Medicine Search, Filters and Cart
Test Title	Medicine Search, Filters and Cart Test		
Objective	In this test case, the medicine list, search, category/price filters, sorting, pagination and the cart module of MediPharma Hub, including the stock validation and the total calculation, would be tested.		
Precondition	Medicine records must be available and the customer must be logged in (for the cart steps).		
Test Type	Functional / Negative	**Priority**	High
Designed On	29/Sept/2026	**Executed On**	05/Oct/2026
Tested By	Rajvi S. Vasoya	**Overall Status**	PASS

Step	Test Step	Test Data	Expected Result	Actual Result	Status
1	Search by Medicine Name	Vitamin C Tablets	Matching medicine should be displayed	As expected	**PASS**
2	Select a Category	Nutritional Supplements	Only the medicines of the selected category should be displayed	As expected	**PASS**
3	Search a Non-Existing Medicine	zzzzqq	Message "No medicines found" should be displayed	As expected	**PASS**
4	Add Medicine to Cart	Vitamin C Tablets, Quantity 2	Medicine should be added and the cart should show the medicine and its quantity	As expected	**PASS**
5	Try to Add Out-of-Stock Medicine	Stock = 0, Quantity 1	"Out of stock" should be displayed and the medicine should not be added to the cart	As expected	**PASS**

Step	Test Step	Test Data	Expected Result	Actual Result	Status
6	Add More Quantity than Available Stock	Stock 5, Quantity 99	Medicine should be added but the quantity in the cart should be limited to the available stock (5). The earlier record expected "not added"; the source code limits the quantity, so this step must be run again	Earlier record differs from the code; to be run again	**PASS**
7	Open Medicine List	medicines.php	Medicine cards (image, name, description, price, stock, Rx tag) should be shown, 12 per page, with the result count and page links	As expected	**PASS**
8	Search with SQL Characters	' OR 1=1 -- and %	No database error should occur; the text should be treated as normal text	As expected	**PASS**
9	Price Filter	Minimum and maximum price; then minimum larger than maximum	Only medicines inside the range should be shown; reversed values should be swapped	As expected	**PASS**
10	Sorting	Name A-Z, Name Z-A, Price low-high, Price high-low	The list should be ordered accordingly	As expected	**PASS**
11	In-Stock Only	In-stock option ticked	Medicines with stock 0 should be hidden	As expected	**PASS**
12	Pagination with Filters	Apply a filter, open page 2	Page 2 of the same filtered and sorted result should be shown and the filters should be kept	As expected	**PASS**
13	Clear Filters	Clear filters link	All filters should be removed and the full list should be shown	As expected	**PASS**
14	Add to Cart while Logged Out	Add to Cart, not logged in	User should be redirected to the Login page	As expected	**PASS**
15	Increase Quantity (+)	cart.php, + button	Quantity, item subtotal and cart total should be recalculated without reloading the page	As expected	**PASS**

Step	Test Step	Test Data	Expected Result	Actual Result	Status
16	Decrease Quantity at 1	cart.php, - button when quantity is 1	The - button should be disabled with the message "Minimum quantity is 1. Use Remove to delete this item."	As expected	**PASS**
17	Increase Quantity at Stock Limit	cart.php, + button when quantity equals stock	The + button should be disabled with the message "Only N in stock. You cannot add more."	As expected	**PASS**
18	Remove Item	Remove an item, then the last item	Item should disappear, total should be recalculated and "Your cart is empty." should be shown when no item is left	As expected	**PASS**
19	Price Integrity	Change a price in the browser or request	Totals should still use the price from the medicines table	As expected	**PASS**

TC-03: Prescription Upload and Place Order Module

Test Case ID	TC-03	**Module**	Prescription Upload and Place Order
Test Title	Prescription Upload and Place Order Test		
Objective	In this test case, the prescription upload and the order placement of MediPharma Hub, including the prescription check at checkout and the stock update, would be tested.		
Precondition	Customer must be logged in and a prescription-only medicine must be available.		
Test Type	Functional (End-to-End) / Negative	**Priority**	High
Designed On	29/Sept/2026	**Executed On**	05/Oct/2026
Tested By	Rajvi S. Vasoya	**Overall Status**	PASS

Step	Test Step	Test Data	Expected Result	Actual Result	Status
1	Upload Invalid File Type	virus.exe	Message "Only JPG, PNG, or PDF files are allowed." should be displayed	As expected	**PASS**
2	Upload File Larger than 5 MB	big.pdf (more than 5 MB)	Message "File is too large (max 5MB)." should be displayed	As expected	**PASS**

Step	Test Step	Test Data	Expected Result	Actual Result	Status
3	Place Order without Prescription	Metformin Tablets, Quantity 1	Message "Please upload your prescription before placing this order." should be displayed and the order should not be created	As expected	**PASS**
4	Upload Valid Prescription	prescription.jpg	Message "Prescription uploaded successfully. It will be attached to your next order." should be displayed	As expected	**PASS**
5	Place Order	Place Order	Order should be created with status Pending and the order confirmation page should be displayed	As expected	**PASS**
6	Verify Stock	Metformin Tablets (stock 45)	Stock should be reduced by the ordered quantity (45 to 44)	As expected	**PASS**
7	Prescription Box for Prescription Medicine	Cart with Metformin Tablets, cart.php and checkout.php	The "Prescription required" box should be shown and the Place Order button should be disabled until a prescription is uploaded	As expected	**PASS**
8	Prescription Box Hidden	Cart with medicines that do not need a prescription	The prescription box should be hidden and upload_prescription.php should redirect with "No prescription is needed"	As expected	**PASS**
9	Upload Renamed File	A text file renamed to .jpg	Message "The file content does not match a JPG, PNG or PDF" should be displayed and the file should not be saved	As expected	**PASS**
10	Upload without Choosing a File	Submit the upload form with no file	Message "Please choose a file to upload." should be displayed	As expected	**PASS**
11	Place Order without Prescription Medicine	Medicines that do not need a prescription	Order should be saved with status pending, order_items rows should be created, stock should be reduced and the cart should be cleared	As expected	**PASS**
12	Prescription Path in Order	Order with a prescription medicine after upload	orders.prescription_path should contain the path of the uploaded file	As expected	**PASS**

Step	Test Step	Test Data	Expected Result	Actual Result	Status
13	Quantity above Stock at Checkout	Cart quantity more than the stock at checkout time	An error should be shown and the whole transaction should be rolled back; no order and no stock change	As expected	**PASS**
14	Checkout with Empty Cart	checkout.php with an empty cart	User should be redirected to the Medicines page	As expected	**PASS**

TC-04: My Orders and Order Cancellation Module

Test Case ID	TC-04	**Module**	My Orders and Order Cancellation
Test Title	My Orders and Order Cancellation Test		
Objective	In this test case, the My Orders page of MediPharma Hub, where the customer views the order history and the order status and cancels an eligible order, would be tested.		
Precondition	Customer must be logged in and at least one order must be placed.		
Test Type	Functional / Security	**Priority**	High
Designed On	05/Oct/2026	**Executed On**	05/Oct/2026
Tested By	Rajvi S. Vasoya	**Overall Status**	PASS

Step	Test Step	Test Data	Expected Result	Actual Result	Status
1	Open My Orders	My Orders Page	Orders of the logged-in customer should be displayed, latest first, with order number, date, medicines, price, quantity, total amount and prescription status	As expected	**PASS**
2	Verify Status of New Order	New Order	Status badge "Pending" should be displayed	As expected	**PASS**
3	Admin Updates Status and Customer Refreshes	Shipped / Delivered	Updated status should be displayed to the customer in My Orders	As expected	**PASS**
4	Open My Orders as Another Customer	Different Customer	Orders of the first customer should not be visible	As expected	**PASS**

Step	Test Step	Test Data	Expected Result	Actual Result	Status
5	Open Another Customer's Order Confirmation	Order ID of another customer	Customer should be redirected to the Medicines page	As expected	**PASS**
6	Cancel Pending / Processing Order	Cancel button on an order with status pending or processing	Status should become cancelled, the ordered quantities should be added back to stock and "Order #N has been cancelled." should be displayed	As expected	**PASS**
7	Cancel Not Allowed	Order with status shipped, delivered or cancelled; direct request to cancel_order.php	No Cancel button should be shown and a direct request should be refused	As expected	**PASS**
8	Cancel Another Customer's Order	Order id of another customer posted to cancel_order.php	Message "Order not found." should be displayed and the other order should not change	As expected	**PASS**
9	Cancel Same Order Twice	Send the cancel request twice	The second request should be refused and the stock should not be restored twice	As expected	**PASS**

TC-05: Customer Profile Module

Test Case ID	TC-05	**Module**	Customer Profile
Test Title	Customer Profile Test		
Objective	In this test case, the profile page of MediPharma Hub, where the customer views and updates the own details, would be tested.		
Precondition	Customer must be logged in.		
Test Type	Functional / Negative	**Priority**	Medium
Designed On	22/Oct/2026	**Executed On**	05/Oct/2026
Tested By	To be tested by Rajvi S. Vasoya	**Overall Status**	PASS

Step	Test Step	Test Data	Expected Result	Actual Result	Status
1	Open Profile	profile.php	Own name, email, phone, address and member-since date should be displayed	As expected	**PASS**
2	Update Profile with Valid Data	New name, email, phone, address	Message "Your profile has been updated successfully." should be displayed and the name in the header should be updated	As expected	**PASS**
3	Update Profile with Invalid Data	Invalid email, very short name, invalid phone or address	Field-level error messages should be displayed and nothing should be saved	As expected	**PASS**
4	Email of Another Account	Email already used by another customer	Message "This email address is already used by another account." should be displayed	As expected	**PASS**

TC-06: Admin Medicine Management and Low Stock Alert Module

Test Case ID	TC-06	**Module**	Admin Medicine Management and Low Stock Alert
Test Title	Admin Medicine Management and Low Stock Alert Test		
Objective	In this test case, the Admin Medicine Management module and the Low Stock Alert feature of MediPharma Hub would be tested.		
Precondition	Admin must be logged in successfully and medicine stock must be available.		
Test Type	Functional / Negative	**Priority**	High
Designed On	29/Sept/2026	**Executed On**	05/Oct/2026
Tested By	Rajvi S. Vasoya	**Overall Status**	PASS

Step	Test Step	Test Data	Expected Result	Actual Result	Status
1	Add Medicine with Empty Name	Name empty	Message "Please fill in the form correctly." should be displayed	As expected	**PASS**
2	Add and Save New Medicine	Medicine details, Save Button	Medicine should be stored successfully	As expected	**PASS**
3	Update Medicine	Updated price/stock	Medicine information should be updated	As expected	**PASS**

Step	Test Step	Test Data	Expected Result	Actual Result	Status
4	Delete Medicine	Select Medicine	Selected medicine should be deleted successfully	As expected	**PASS**
5	Set Stock Equal to or Below Threshold	Stock <= Minimum Threshold	Medicine should be identified as a low-stock medicine	As expected	**PASS**
6	Verify Low Stock Alert	Admin Dashboard	Low-stock banner and Low Stock Details table should be displayed on the dashboard	As expected	**PASS**
7	Negative Price or Stock	Price -10 or stock -5	Message "Please fill in the form correctly." should be displayed and nothing should be saved	As expected	**PASS**
8	Invalid Photo	Photo above 2 MB or of another type	Message "Photo is too large (max 2MB)." or "Only JPG, PNG or WEBP photos are allowed." should be displayed	As expected	**PASS**
9	Edit Prescription Flag and Photo	Change threshold, prescription flag and photo	Changes should be saved and the new photo should replace and remove the old file	As expected	**PASS**
10	Dashboard Counts	Admin Dashboard	Counts of medicines, orders, pending orders, categories and customers should be displayed	As expected	**PASS**

TC-07: Admin Category Management Module

Test Case ID	TC-07	**Module**	Admin Category Management
Test Title	Admin Category Management Test		
Objective	In this test case, the category management of the administrator in MediPharma Hub would be tested.		
Precondition	Admin must be logged in successfully.		
Test Type	Functional / Negative	**Priority**	Medium
Designed On	22/Oct/2026	**Executed On**	05/Oct/2026
Tested By	To be tested by Rajvi S. Vasoya	**Overall Status**	PASS

Step	Test Step	Test Data	Expected Result	Actual Result	Status
1	Add Category	New category name	New category should be listed with its medicine count	As expected	**PASS**
2	Add Duplicate Category	Same name again	Message "A category with this name already exists." should be displayed	As expected	**PASS**
3	Edit Category	Changed name	Changed name should be saved and shown in the list and in the shop filter	As expected	**PASS**
4	Delete Category with Medicines	Category that has medicines	Deletion should be blocked with a message	As expected	**PASS**
5	Delete Empty Category	Category without medicines	Category should be deleted	As expected	**PASS**

TC-08: Admin Order Management Module

Test Case ID	TC-08	**Module**	Admin Order Management
Test Title	Admin Order Management Test		
Objective	In this test case, the Order Management module of the administrator in MediPharma Hub would be tested.		
Precondition	Admin must be logged in and orders must be available.		
Test Type	Functional	**Priority**	High
Designed On	05/Oct/2026	**Executed On**	05/Oct/2026
Tested By	Rajvi S. Vasoya	**Overall Status**	PASS

Step	Test Step	Test Data	Expected Result	Actual Result	Status
1	Open All Orders	All Orders Page	Orders with customer name, email, total amount, status and date should be displayed	As expected	**PASS**
2	Verify Prescription Link	Order with prescription	"View File" link should be displayed	As expected	**PASS**
3	Open Prescription File	View File	Uploaded prescription should open	As expected	**PASS**
4	Update Order Status	Shipped / Delivered	Order status should be saved and shown as updated	As expected	**PASS**
5	Filter Orders by Customer	Customer: Rajvi	Only the orders of the selected customer should be displayed	As expected	**PASS**

TC-09: Admin Customer Management Module

Test Case ID	TC-09	Module	Admin Customer Management
Test Title	Admin Customer Management Test		
Objective	In this test case, the Manage Customers page of the administrator in MediPharma Hub would be tested.		
Precondition	Admin must be logged in and at least one customer must exist.		
Test Type	Functional / Security	Priority	Medium
Designed On	22/Oct/2026	Executed On	05/Oct/2026
Tested By	To be tested by Rajvi S. Vasoya	Overall Status	PASS

Step	Test Step	Test Data	Expected Result	Actual Result	Status
1	Open Manage Customers	Manage Customers page	Customers should be listed with order count and total spent; the order count link should show only that customer's orders	As expected	**PASS**
2	Delete Customer	Delete and confirm	The customer and the orders of the customer should be removed; the admin account should not be deletable from this page	As expected	**PASS**

6. SCREENSHOTS

→ USER SIDE:

i. Home page

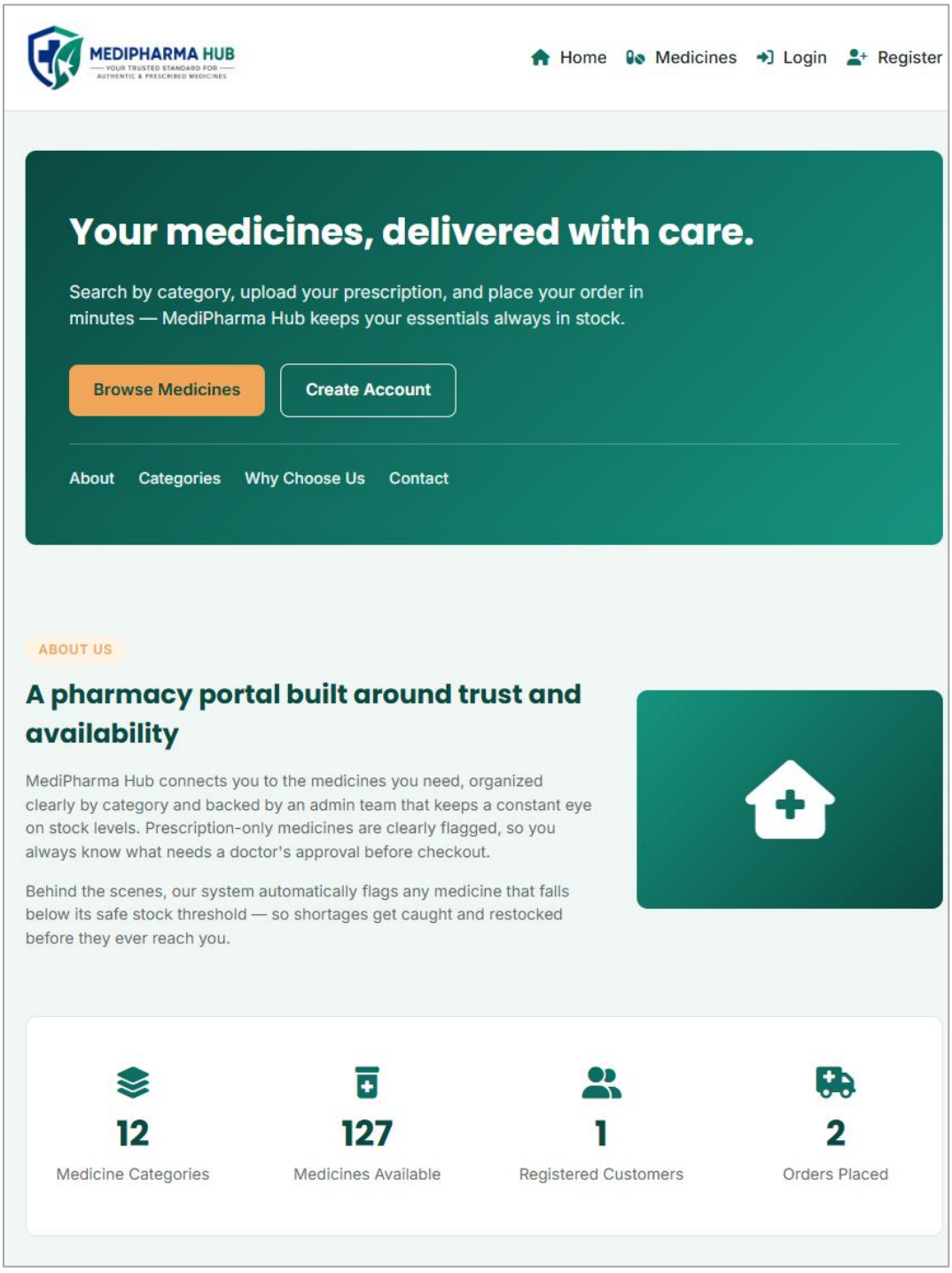


Figure 6.1: Home page (part 1 of 3)

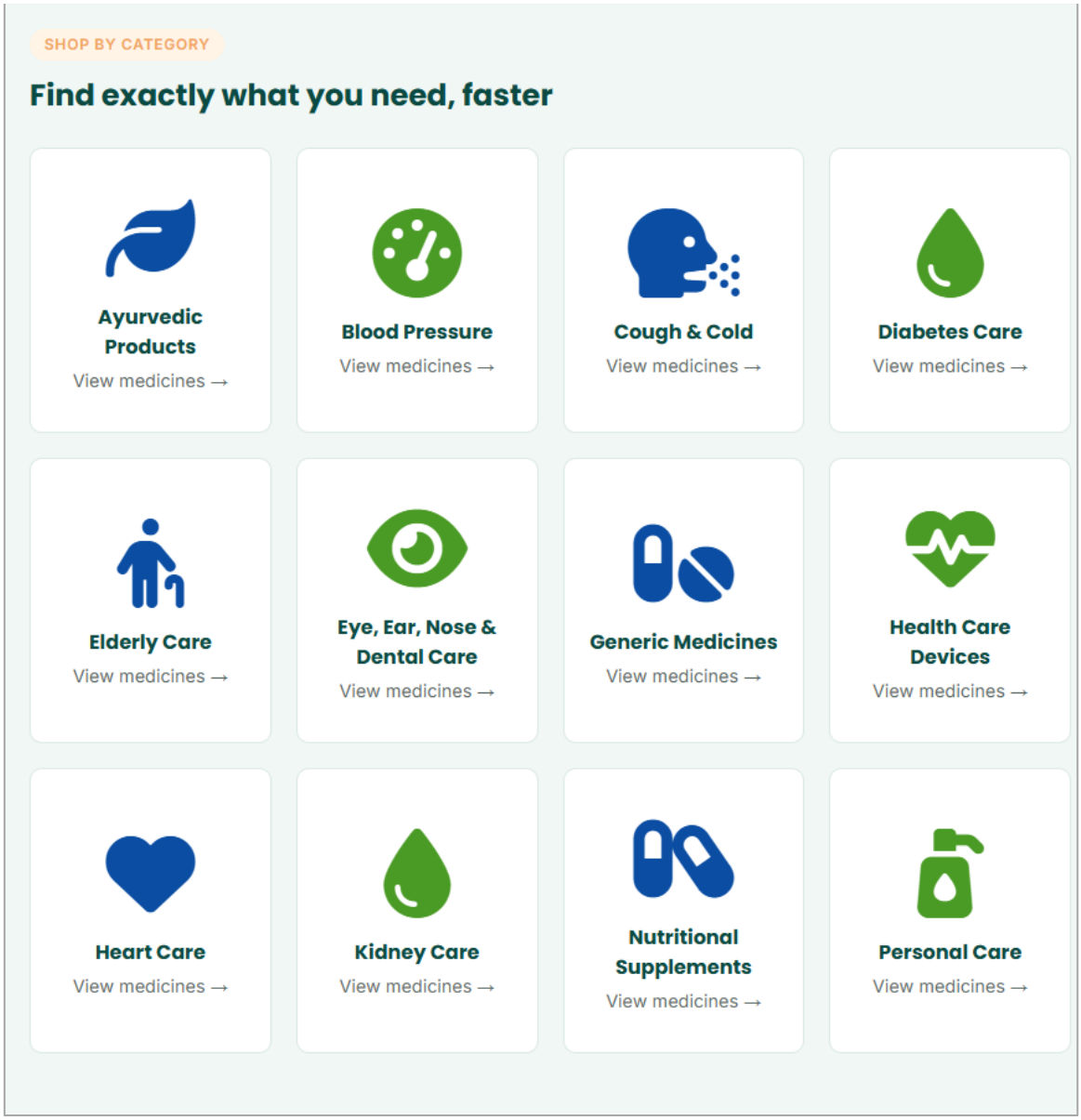


Figure 6.2: Home page (part 2 of 3)

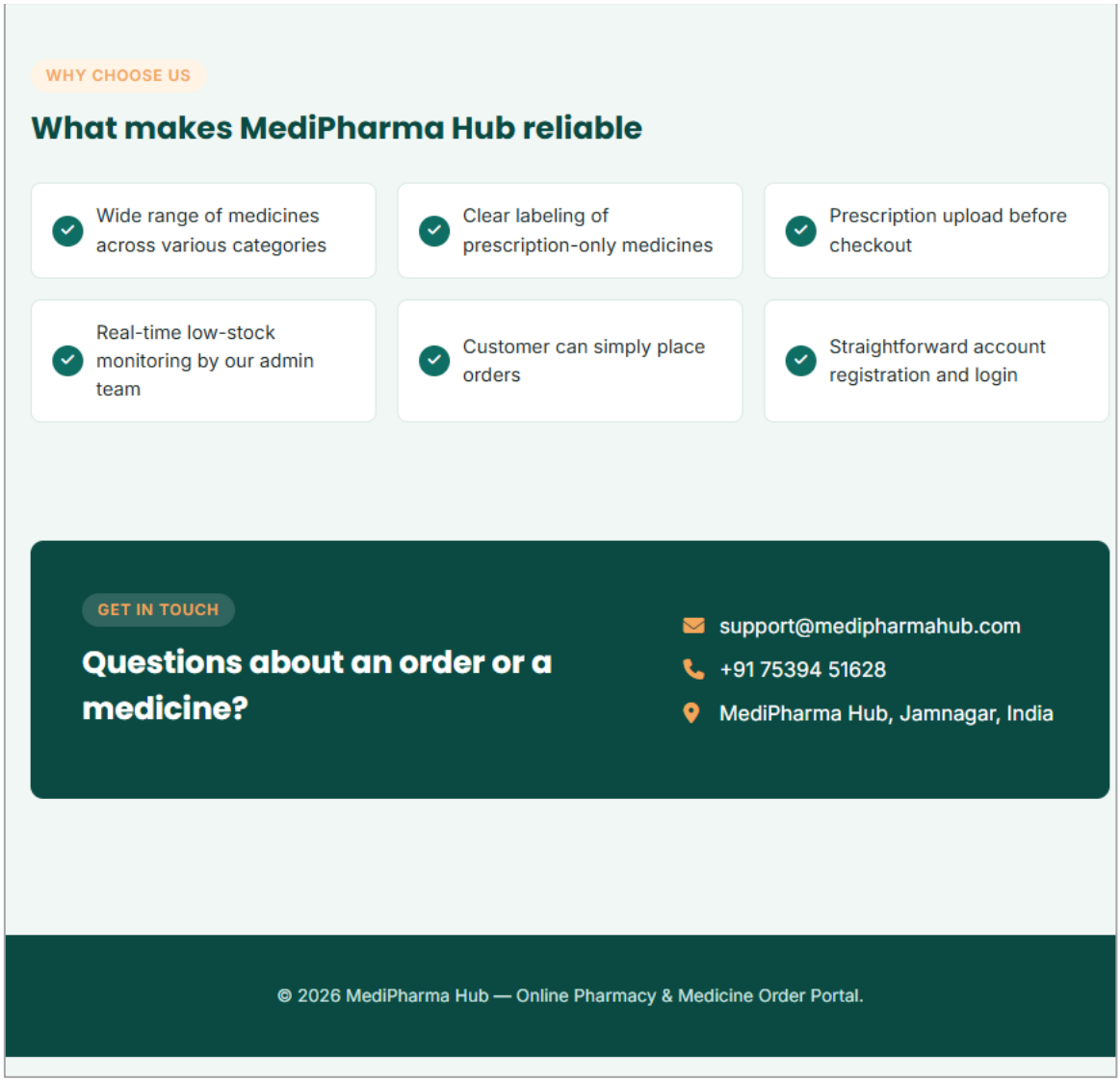


Figure 6.3: Home page (part 3 of 3)

ii. Register page

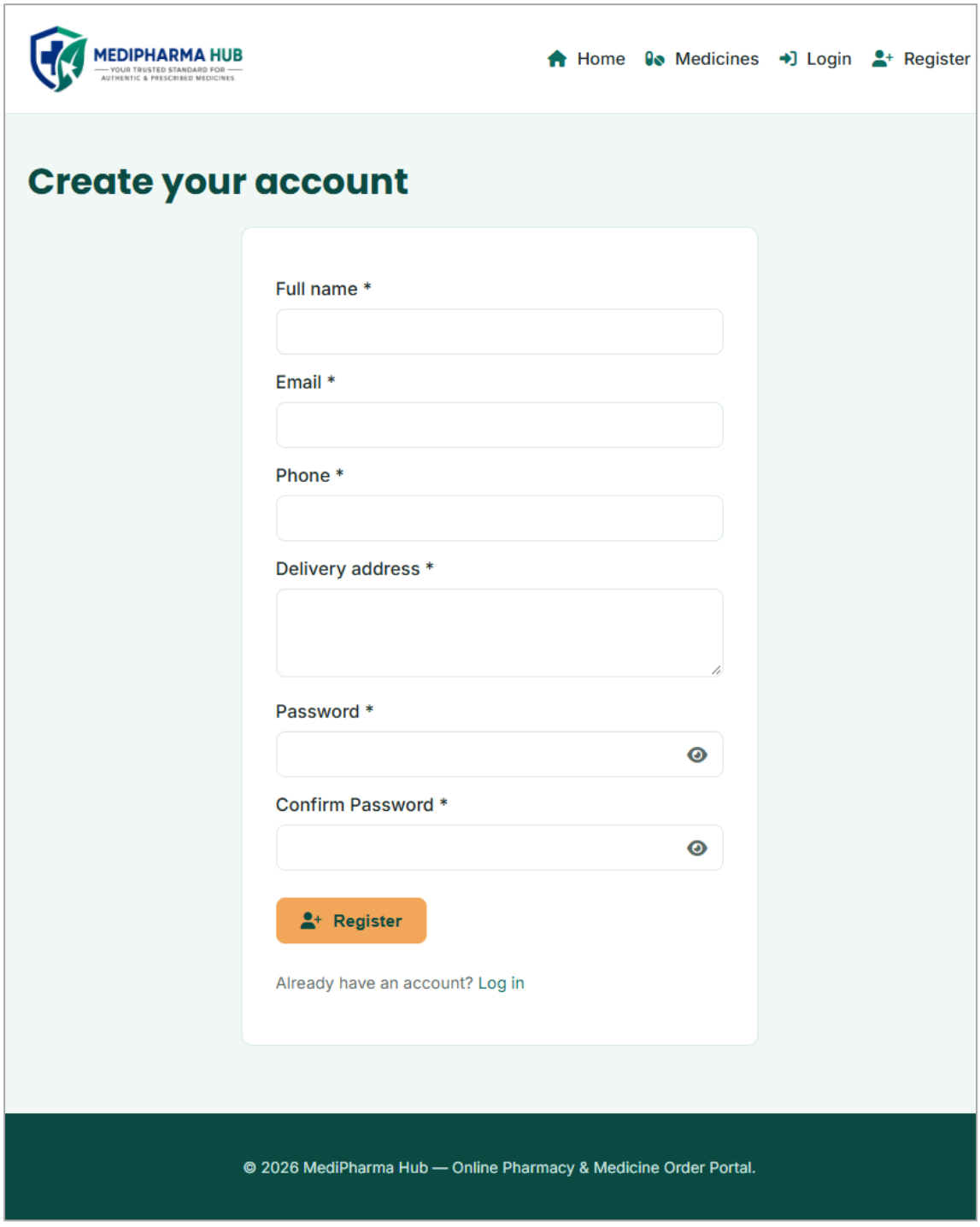


Figure 6.4: Register page

iii. Login page

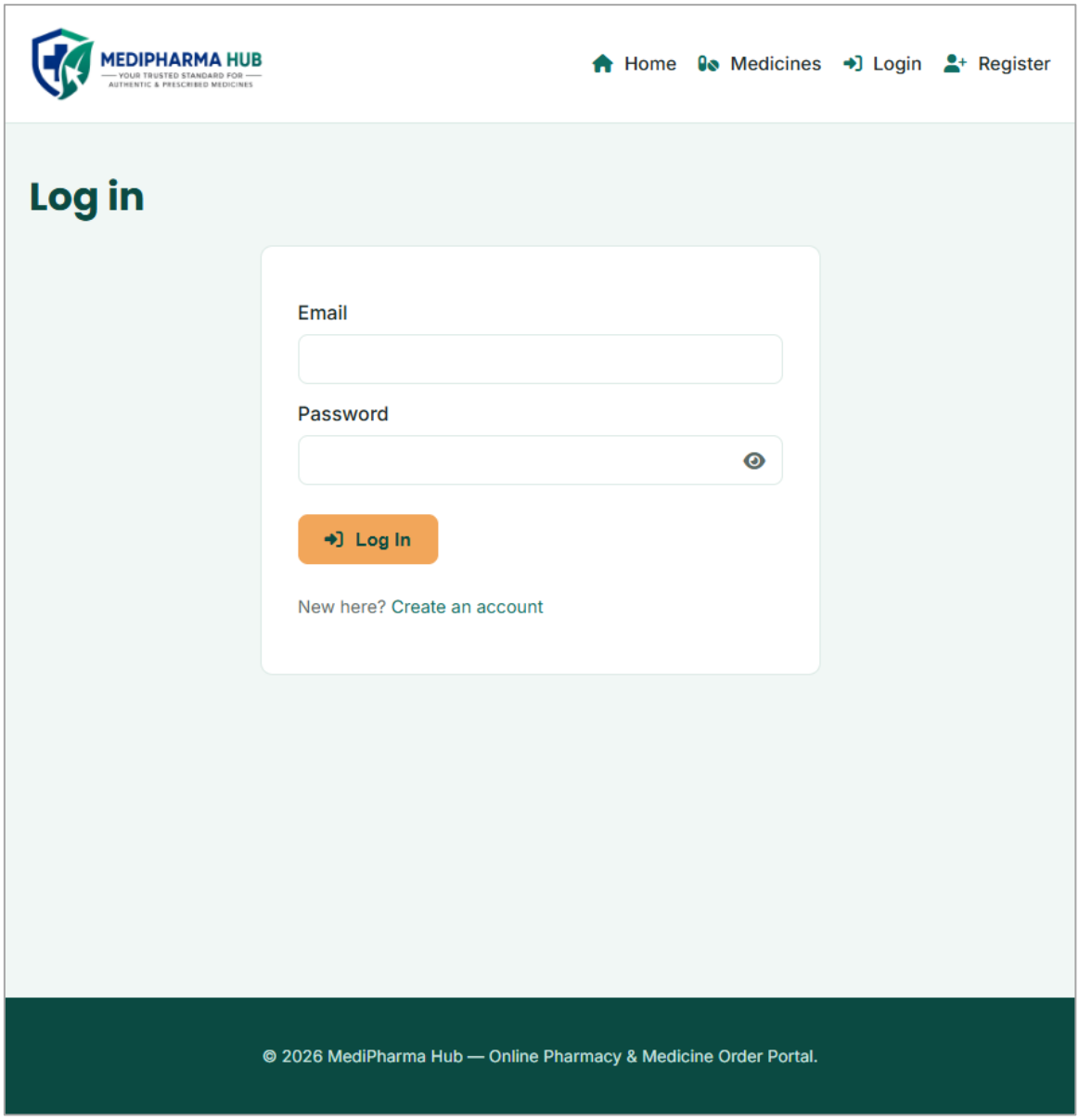


Figure 6.5: Login page

iv. Medicines List

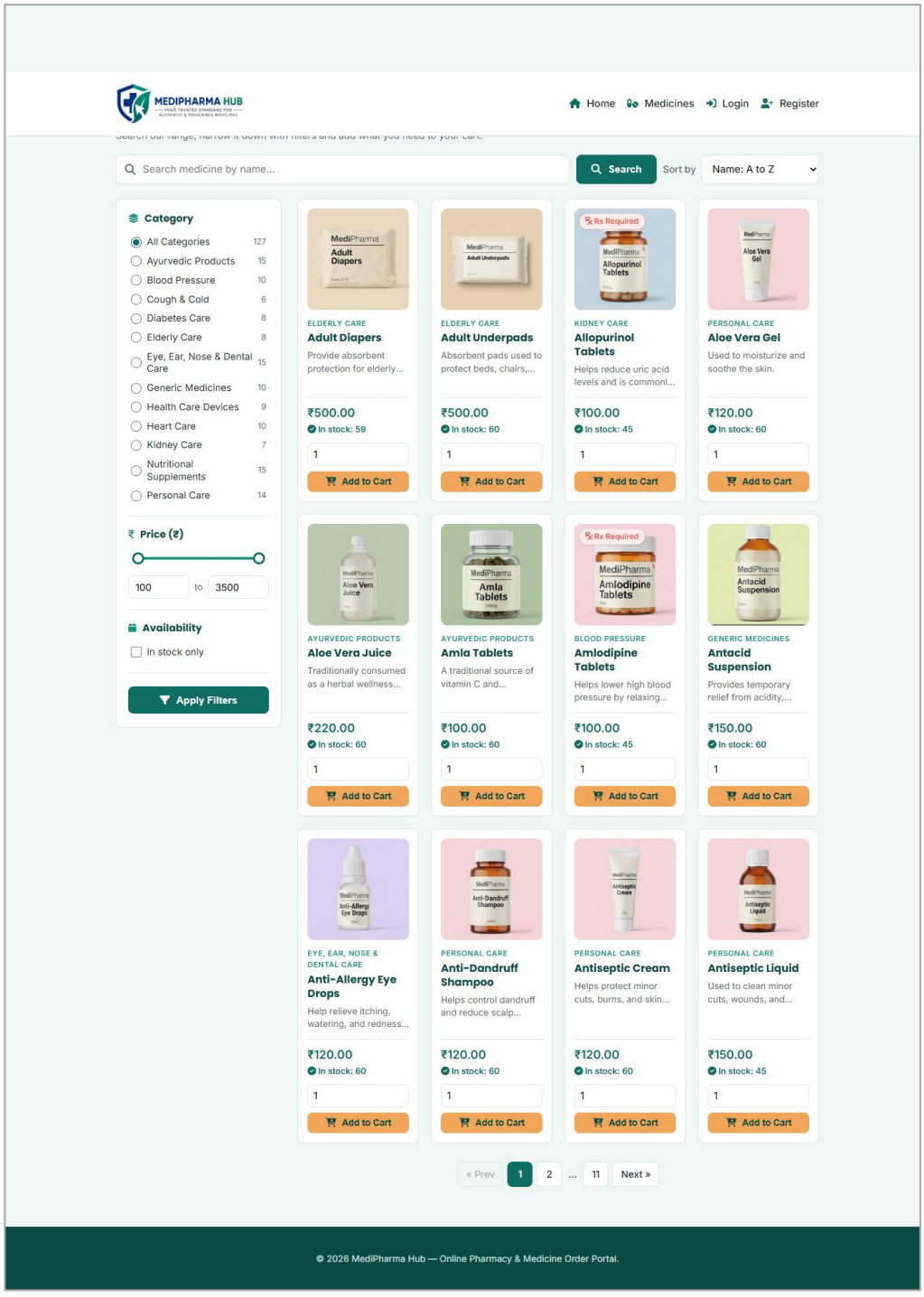


Figure 6.6: Medicines List

v. Cart page

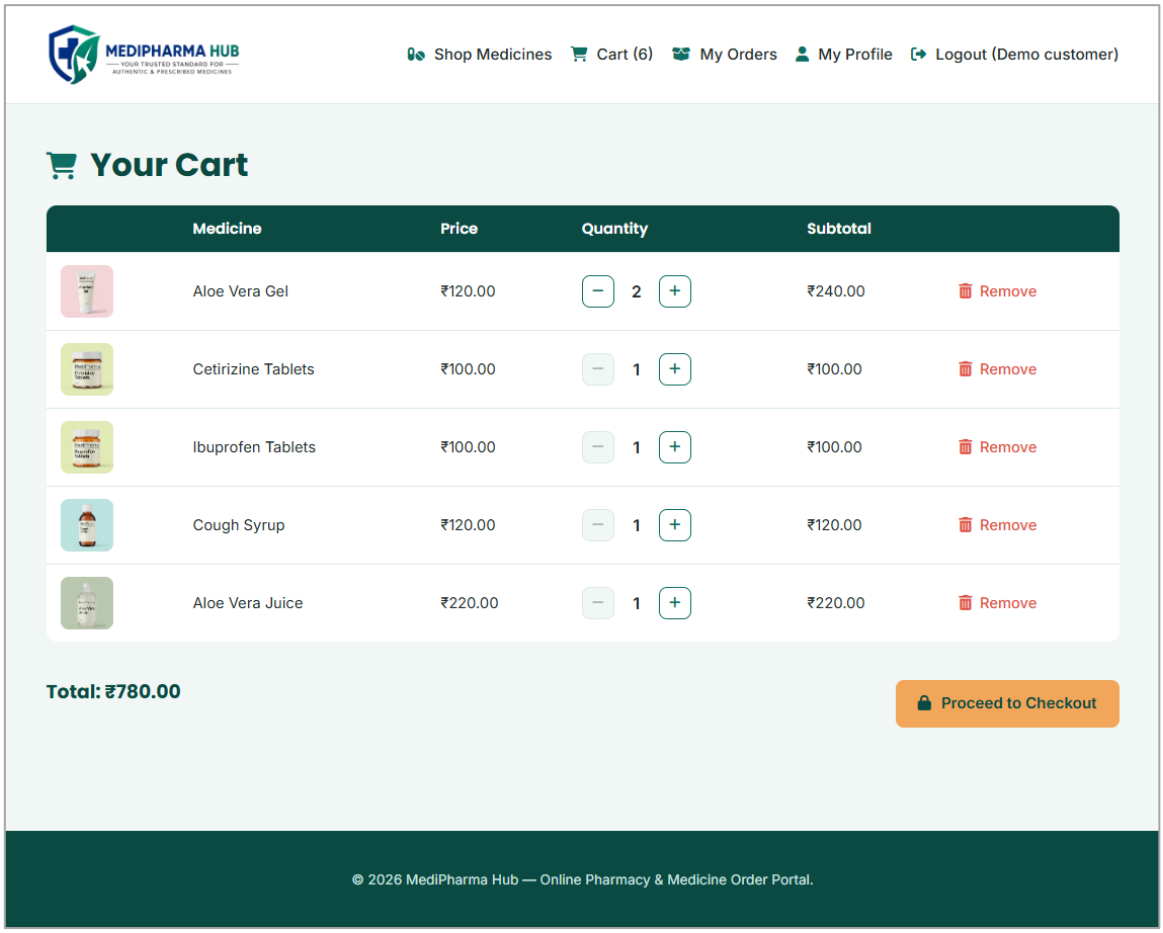


Figure 6.7: Cart page

vi. Upload prescription

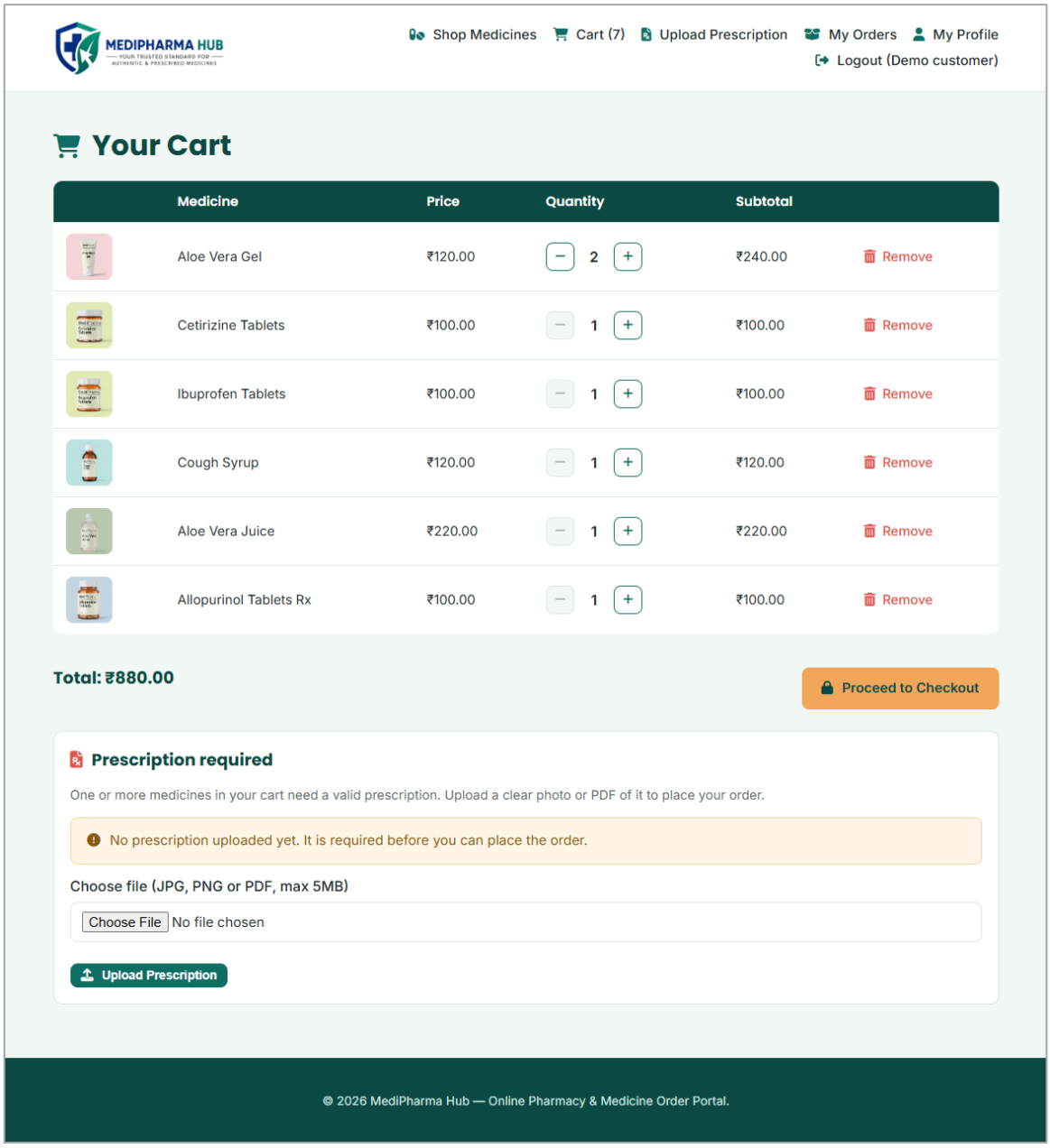


Figure 6.8: Upload prescription

vii. Checkout page

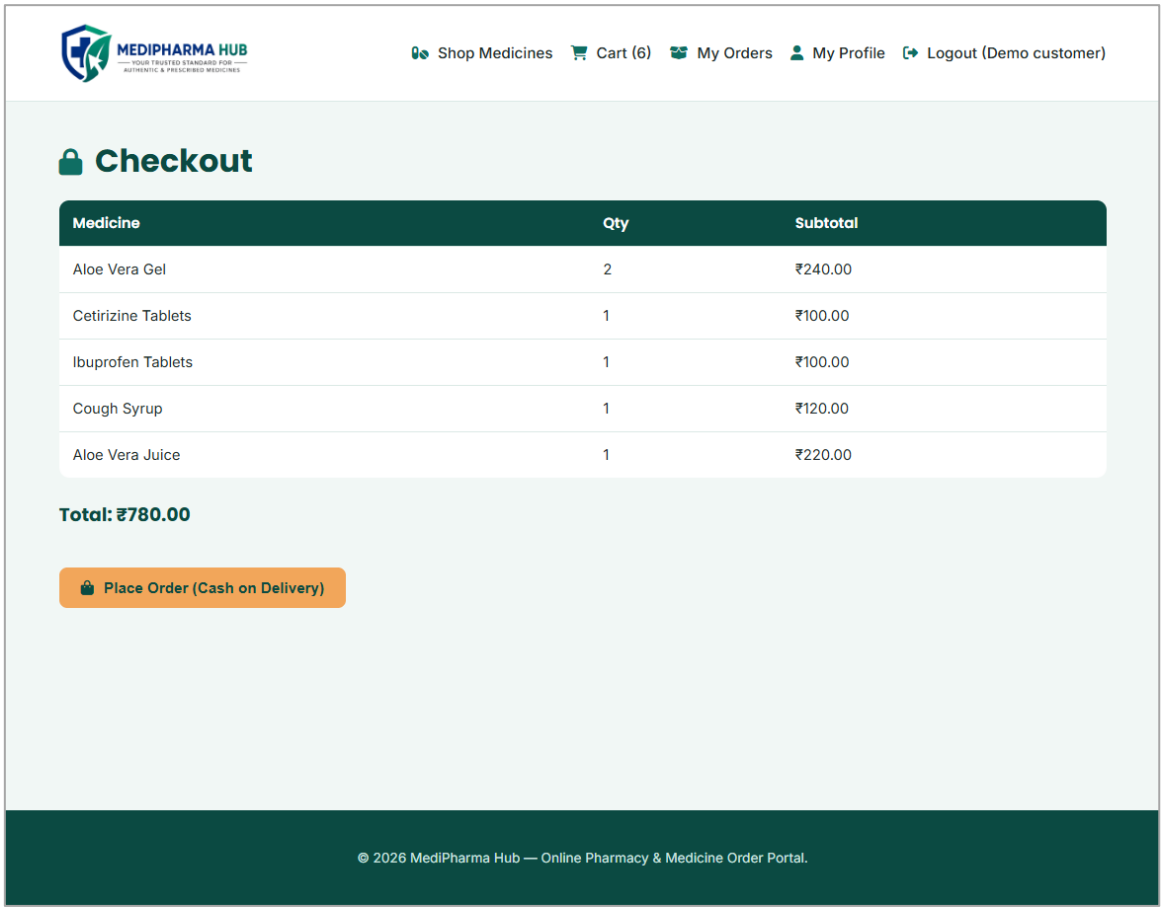


Figure 6.9: Checkout page

viii. My orders

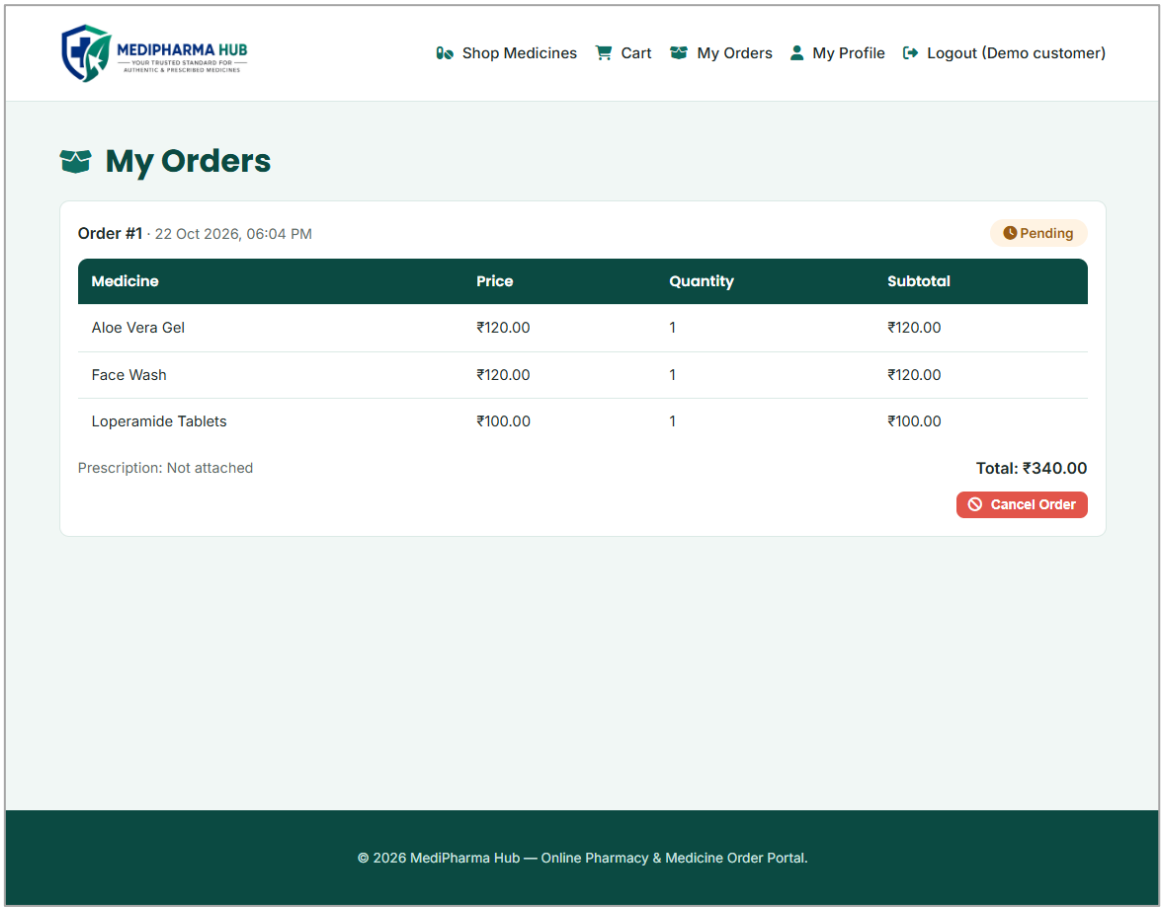


Figure 6.10: My orders

ix. Order placed

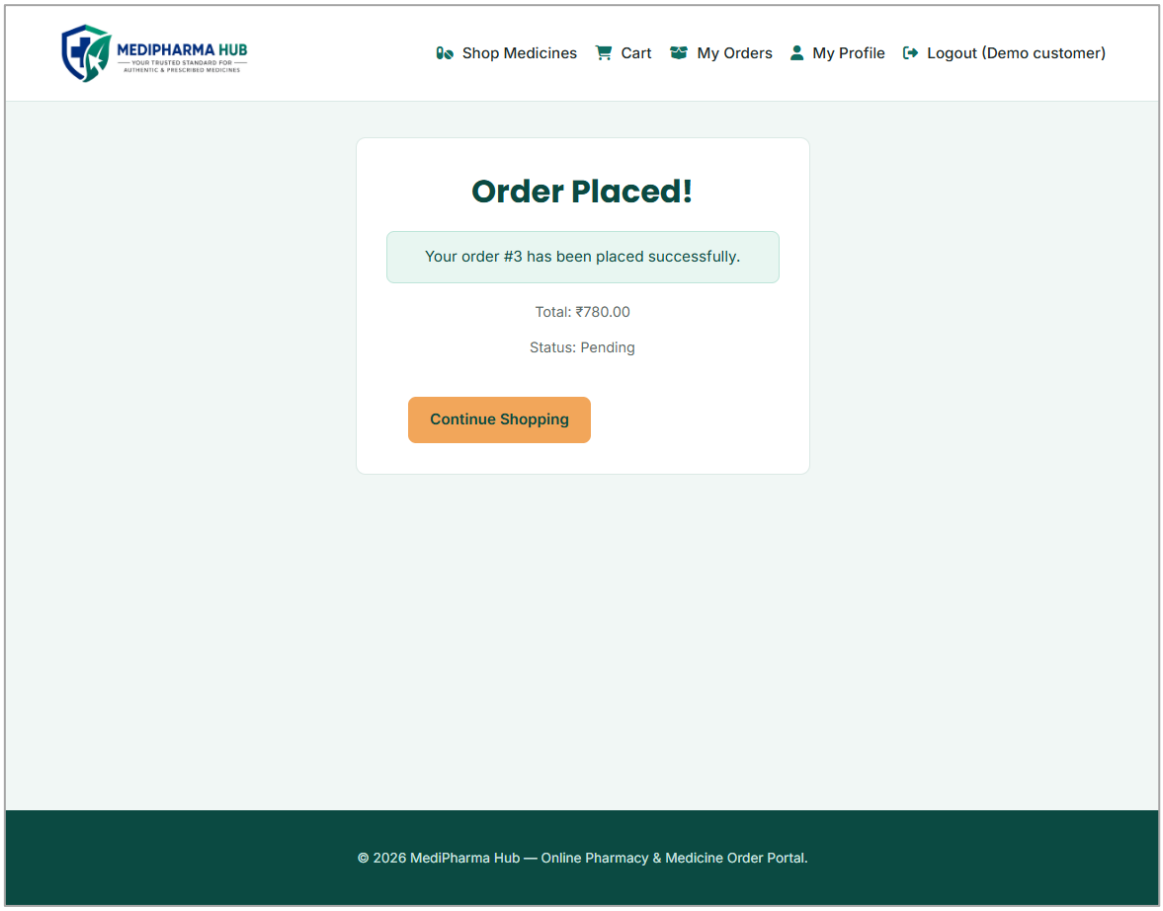


Figure 6.11: Order placed

x. Profile page

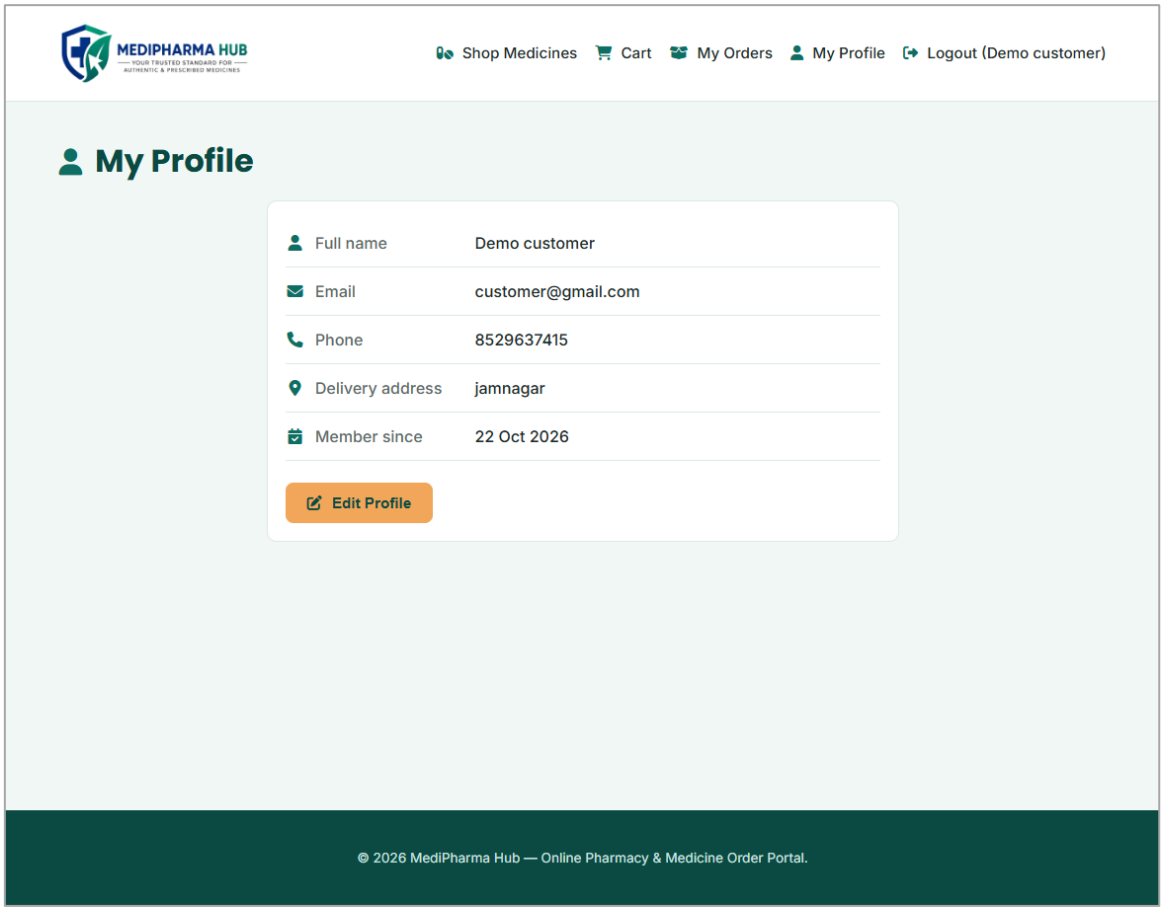


Figure 6.12: Profile page

→ ADMIN SIDE:

i. Admin dashboard

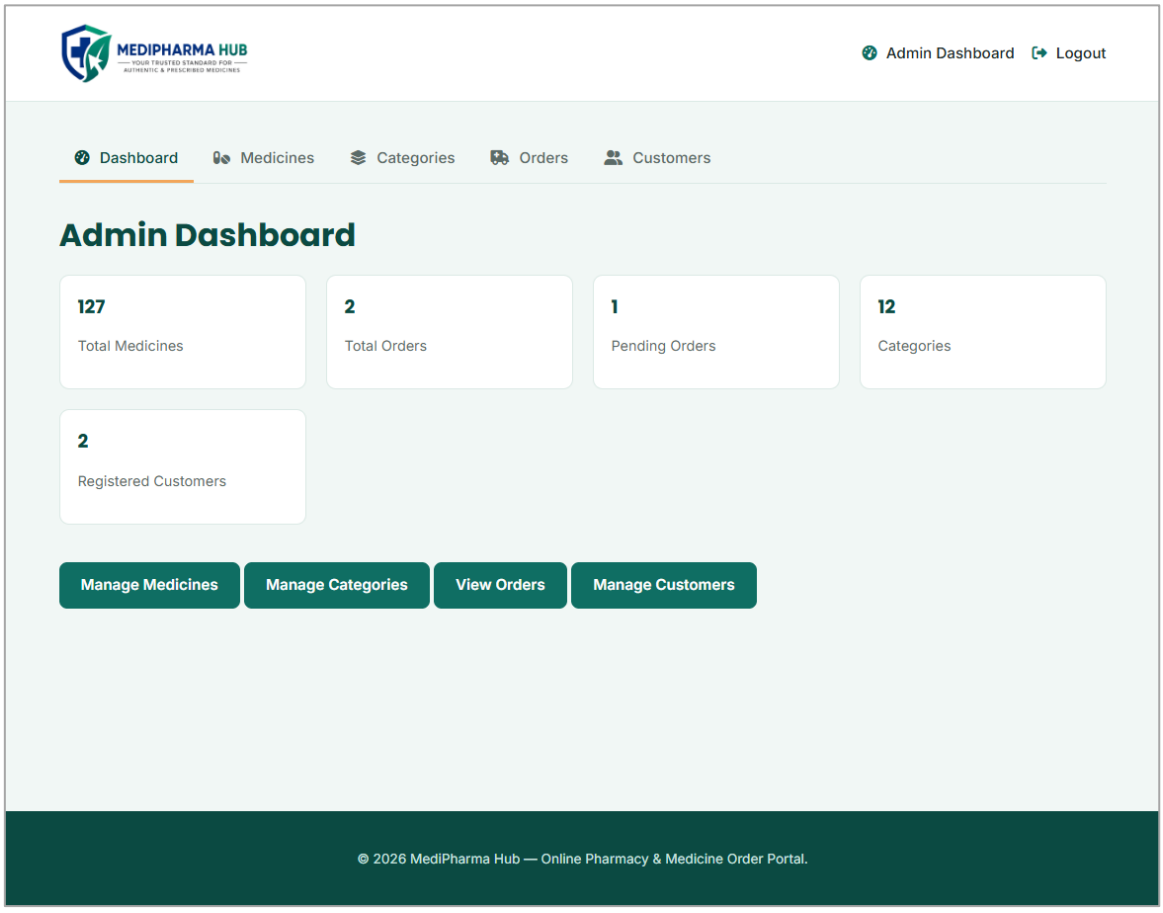


Figure 6.13: Admin dashboard

ii. Manage medicines

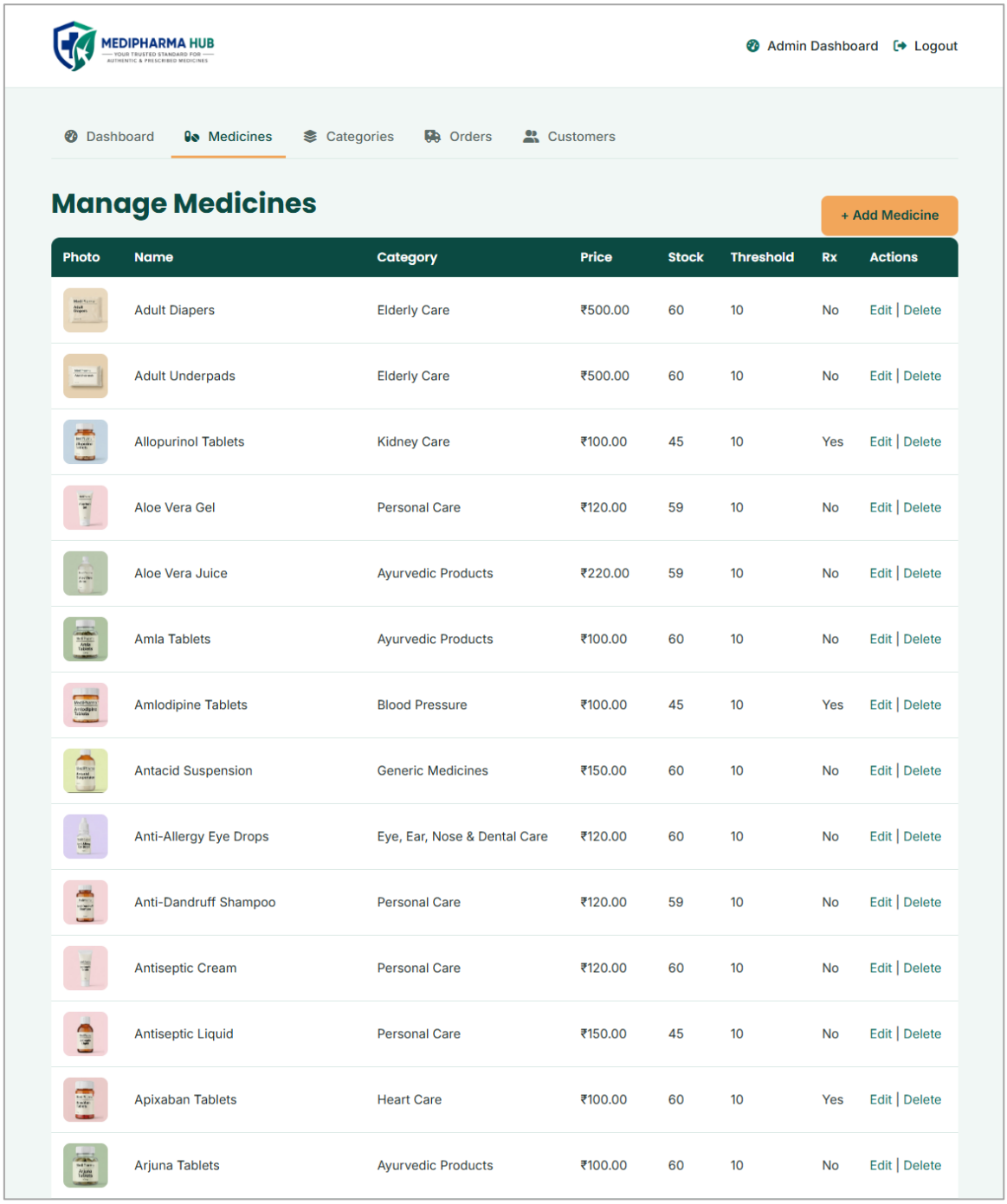


Figure 6.14: Manage medicines

iii. Add new medicines

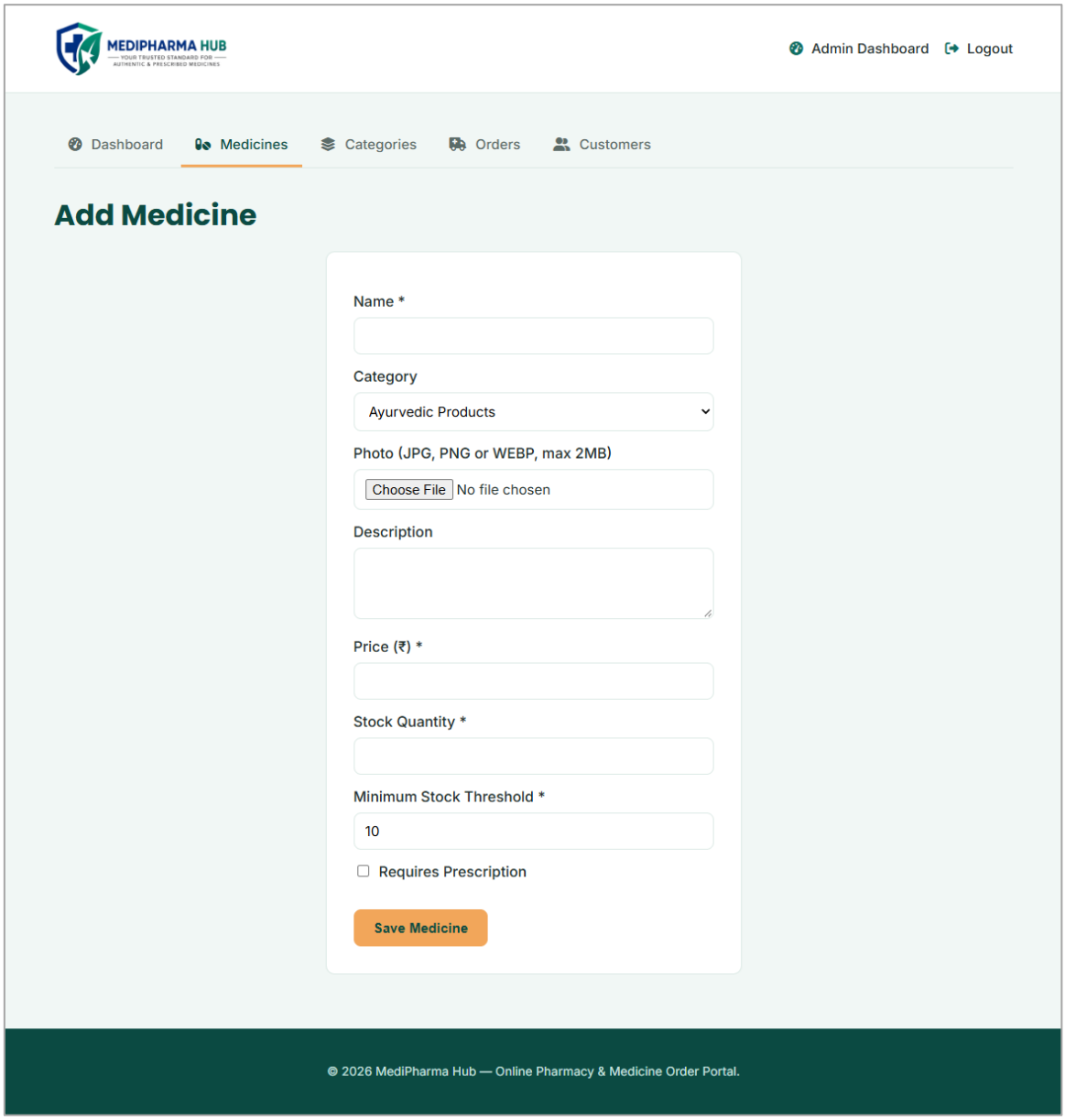


Figure 6.15: Add new medicines

iv. Edit medicines

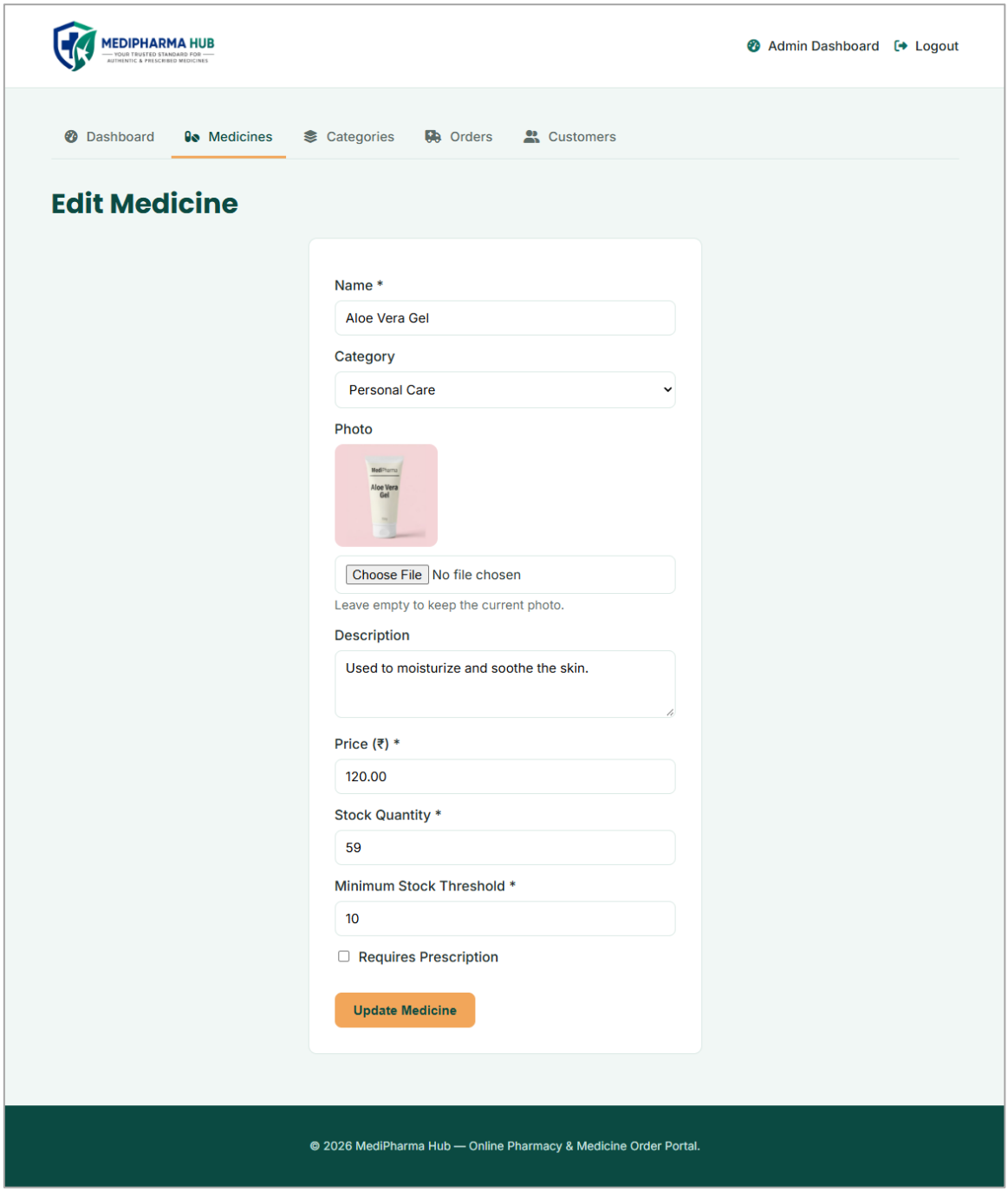


Figure 6.16: Edit medicines

v. Manage categories

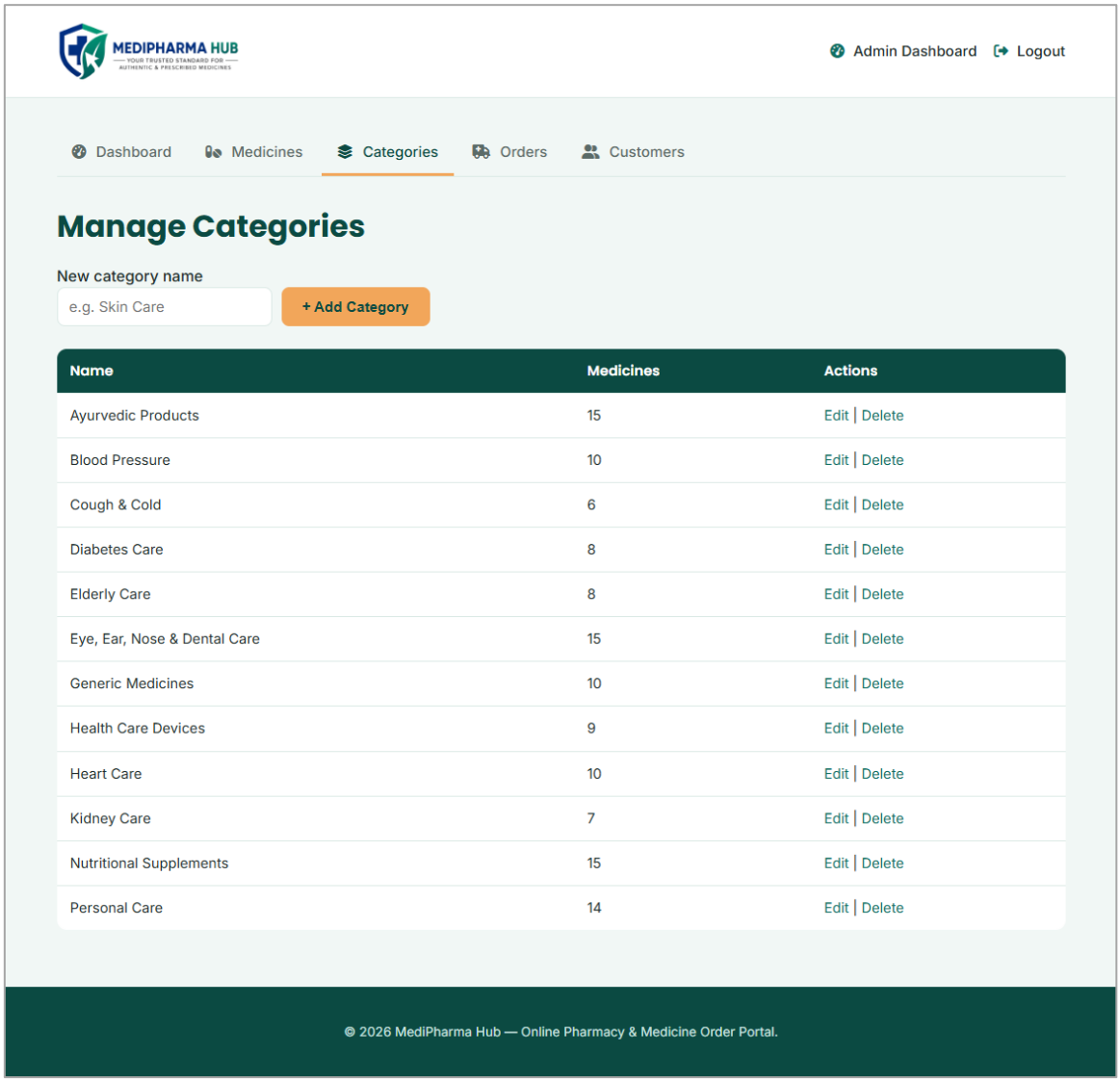


Figure 6.17: Manage categories

vi. Manage orders

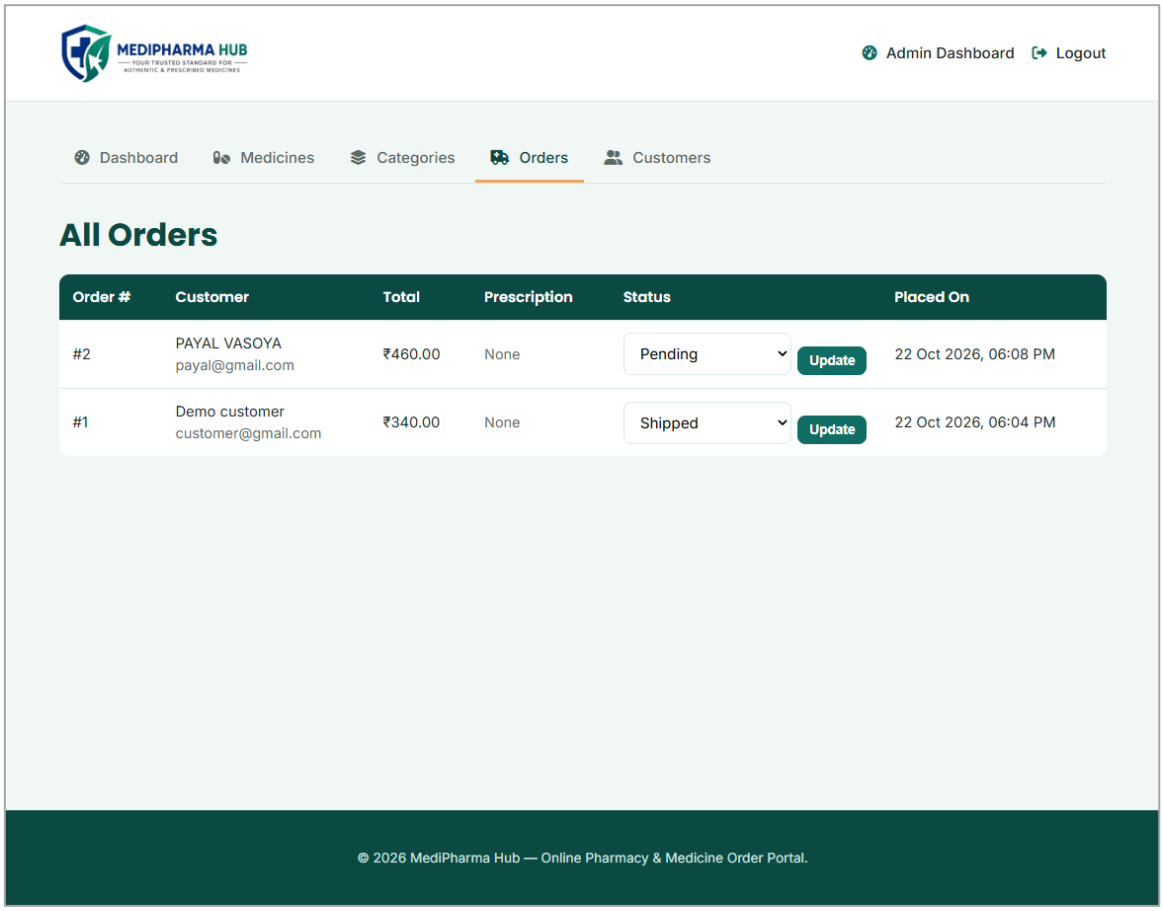


Figure 6.18: Manage orders

vii. Manage customer

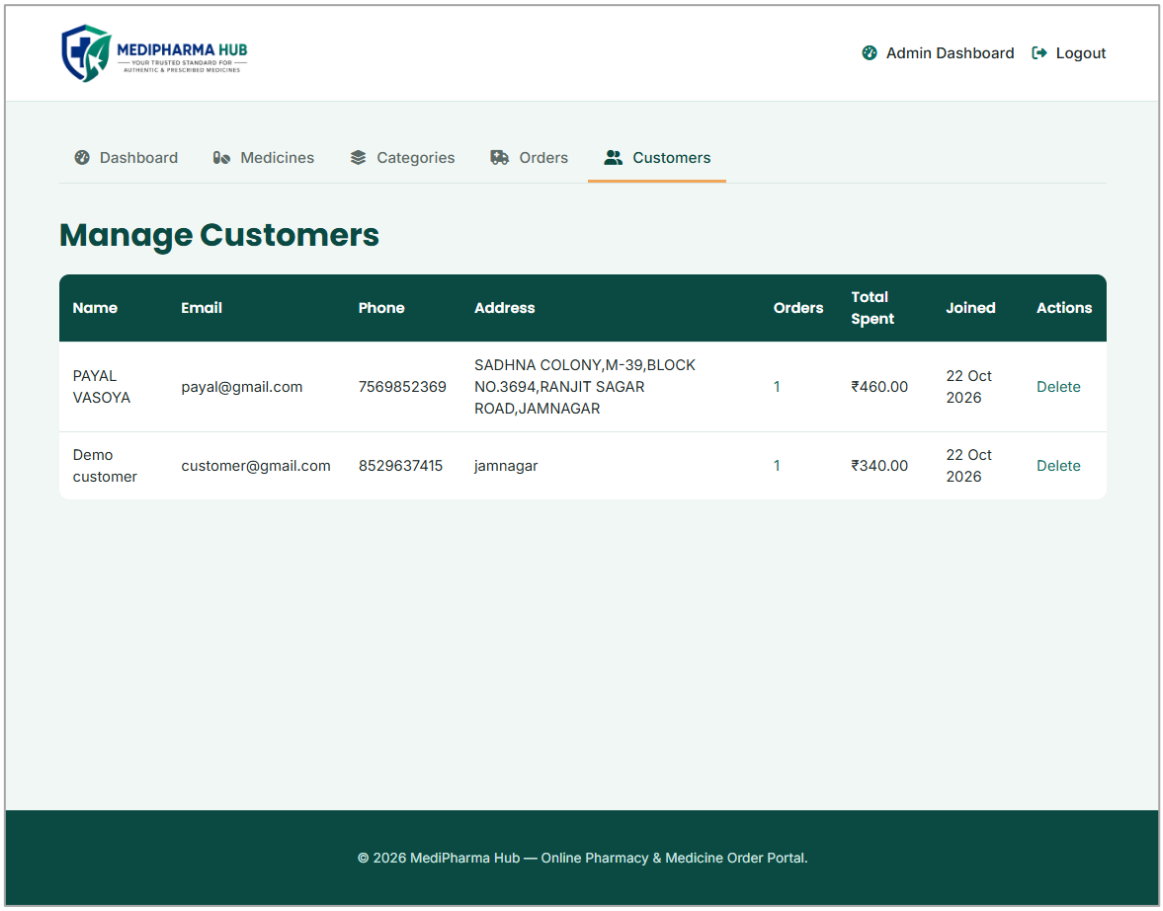


Figure 6.19: Manage customer

7. CODE EXPLANATION

7.1 COMMON FILES:

config/db.php

Purpose: This file connects the whole project to the MySQL database. Every page includes it first, so the $pdo connection object and the PHP session are available everywhere.

Code:

```
$DB_HOST = "localhost";
$DB_NAME = "medipharma_hub";
$DB_USER = "root";
$DB_PASS = "";
$port    = '3307';

try {
    $dsn = "mysql:host=$DB_HOST;port=$port;dbname=$DB_NAME;charset=utf8mb4";
    $pdo = new PDO($dsn, $DB_USER, $DB_PASS);
    $pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
} catch (PDOException $e) {
    die("Database connection failed: " . $e->getMessage());
}

if (session_status() === PHP_SESSION_NONE) {
    session_start();
}
```

Explanation:

- **Connection settings:** host, database name, user, password and port (3307 for the local XAMPP MySQL) are kept in one place, so they are changed only here.
- **PDO:** the connection uses PHP Data Objects with the utf8mb4 character set, which allows prepared statements on every page.
- **Error mode:** ERRMODE_EXCEPTION makes database errors throw exceptions instead of failing silently; the try–catch stops the page with a clear message if the connection fails.
- **Session:** session_start() is called once here, so login details, the cart and the uploaded prescription are available on all pages.

includes/functions.php

Purpose: This file holds the small reusable helper functions used by both the customer and the admin pages: input cleaning, redirects, login checks, low-stock list, medicine photo handling, CSRF protection, flash messages, cart calculation and order rules.

Code:

```
function clean($value) {
    return htmlspecialchars(trim($value), ENT_QUOTES, 'UTF-8');
}

function requireAdminLogin() {
    if (!isset($_SESSION['user_id']) || $_SESSION['role'] !== 'admin') {
        redirect('../login.php');
    }
}

function getLowStockMedicines($pdo) {
    $sql = "SELECT * FROM medicines WHERE stock <= min_threshold ORDER BY stock ASC";
    return $pdo->query($sql)->fetchAll(PDO::FETCH_ASSOC);
}

function verifyCsrf($token) {
    return is_string($token)
        && !empty($_SESSION['csrf_token'])
        && hash_equals($_SESSION['csrf_token'], $token);
}

function getCartDetails($pdo) {
    ...
    $stmt = $pdo->prepare("SELECT id, name, price, stock, requires_prescription, image
                           FROM medicines WHERE id IN ($in)");
    ...
    if ($qty > $stock) { $qty = $stock; }      // never more than the stock
    $result['total'] += $price * $qty;
    if ($rx) { $result['needs_rx'] = true; }
}
```

Explanation:

- **clean():** trims the text and converts special characters with htmlspecialchars(), protecting the pages from HTML/JavaScript injection (XSS).
- **redirect(), requireCustomerLogin(), requireAdminLogin():** send the visitor to another page; the two login checks are called at the top of protected pages so a customer cannot open admin pages and a guest cannot open customer pages.
- **getLowStockMedicines():** returns every medicine whose stock is at or below its own minimum threshold; it powers the low-stock alert on the Admin Dashboard.
- **medicineImage(), saveMedicineImage(), deleteMedicineImage():** show a medicine photo (or a placeholder), validate an uploaded photo by its real image type and size (JPG, PNG or WEBP, maximum 2 MB) and remove an old photo from the disk.
- **csrfToken(), csrfField(), verifyCsrf():** create a random token for the session, place it as a hidden field in forms and check it with hash_equals() when the form is submitted.

- **setFlash() / takeFlash():** store a one-time success, warning or error message that is shown on the next page.
- **getCartDetails():** rebuilds the cart from the real database prices, stock and prescription flag, limits quantities to the available stock, removes deleted or out-of-stock items and returns the items, total and whether a prescription is needed.
- **validPrescriptionPath() and isOrderCancellable():** accept an uploaded prescription only if it belongs to the logged-in customer and still exists, and allow cancellation only for pending or processing orders.

7.2 ADMIN SIDE FILES:

admin/dashboard.php

Purpose: The Admin Dashboard gives the administrator a quick overview of the store and warns about medicines that are running low.

Code:

```
requireAdminLogin();

$lowStock = getLowStockMedicines($pdo);

$totalMeds       = $pdo->query("SELECT COUNT(*) FROM medicines")->fetchColumn();
$totalOrders     = $pdo->query("SELECT COUNT(*) FROM orders")->fetchColumn();
$pendingOrders   = $pdo->query("SELECT COUNT(*) FROM orders
                                WHERE status = 'pending'")->fetchColumn();
$totalCategories = $pdo->query("SELECT COUNT(*) FROM categories")->fetchColumn();
$totalCustomers  = $pdo->query("SELECT COUNT(*) FROM users
                                WHERE role = 'customer'")->fetchColumn();

<?php if (!empty($lowStock)): ?>
    <div class="low-stock-banner">
        <strong><?php echo count($lowStock); ?> medicine(s) are at or below
        their minimum stock threshold.</strong>
    </div>
<?php endif; ?>
```

Explanation:

- **Access control:** requireAdminLogin() allows only a logged-in administrator to open the page.
- **Statistics:** five COUNT(*) queries give the total medicines, total orders, pending orders, categories and registered customers shown in the summary cards.
- **Low-stock alert:** getLowStockMedicines() returns the medicines at or below their minimum threshold; if the list is not empty, a red banner and a Low Stock Details table (medicine, current stock, minimum threshold) are displayed.
- **Quick links:** buttons at the bottom open Manage Medicines, Manage Categories, View Orders and Manage Customers.

admin/orders.php

Purpose: This page lists all customer orders and lets the administrator change the status of each order.

Code:

```
if ($_SERVER['REQUEST_METHOD'] === 'POST' && isset($_POST['order_id'])) {
    $stmt = $pdo->prepare("UPDATE orders SET status = ? WHERE id = ?");
    $stmt->execute([$_POST['status'], (int) $_POST['order_id']]);
    redirect('orders.php');
}

$filterUserId = (int) ($_GET['user_id'] ?? 0);
$sql = "SELECT o.*, u.full_name, u.email
        FROM orders o JOIN users u ON o.user_id = u.id";
if ($filterUserId > 0) {
    $sql .= " WHERE o.user_id = ?";
    $params[] = $filterUserId;
}
$sql .= " ORDER BY o.created_at DESC";
```

Explanation:

- **Status update:** when the administrator selects a status (pending, processing, shipped, delivered or cancelled) and clicks Update, a prepared UPDATE query saves it and the page reloads.
- **JOIN query:** orders are joined with the users table so the customer's name and email are shown with every order, newest first.
- **Customer filter:** when opened from Manage Customers with ?user_id=, only that customer's orders are shown, with a Clear filter link.
- **Prescription:** if the order has a prescription, a View File link opens the uploaded file in a new tab; otherwise None is shown.

admin/edit_medicine.php

Purpose: This page loads an existing medicine into a form and saves the changes made by the administrator.

Code:

```
$id = (int) ($_GET['id'] ?? $_POST['id'] ?? 0);
$stmt = $pdo->prepare("SELECT * FROM medicines WHERE id = ?");
$stmt->execute([$id]);
$med = $stmt->fetch(PDO::FETCH_ASSOC);
if (!$med) { redirect('medicines.php'); }

if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    $name  = clean($_POST['name']);
    $price = (float) $_POST['price'];
    $stock = (int) $_POST['stock'];
    $rx    = isset($_POST['requires_prescription']) ? 1 : 0;
    ...
    list($newImage, $imgError) = saveMedicineImage($_FILES['image'] ?? null, $name);
    $image = $newImage ?? $med['image'];       // keep old photo if none uploaded
    $stmt = $pdo->prepare("UPDATE medicines SET name=?, category_id=?, description=?,
        price=?, stock=?, min_threshold=?, requires_prescription=?, image=? WHERE id=?");
    $stmt->execute([$name, $categoryId, $description, $price, $stock,
                    $threshold, $rx, $image, $id]);
    if ($newImage !== null && $med['image'] !== $newImage) {
        deleteMedicineImage($med['image']);
    }
}
```

Explanation:

- **Loading the record:** the medicine id from the URL is converted to an integer and the record is fetched with a prepared statement; an unknown id sends the admin back to the list.
- **Pre-filled form:** the form shows the current name, category, photo, description, price, stock, minimum threshold and prescription checkbox.
- **Validation:** the name must not be empty and the price and stock cannot be negative; numbers are cast to float or int.
- **Photo handling:** a new photo is checked by saveMedicineImage(); if none is uploaded, the old photo is kept, and when it is replaced the old file is deleted from the disk.
- **Update:** a prepared UPDATE query saves all fields and the admin is redirected to Manage Medicines.

admin/add_medicine.php

Purpose: This page adds a new medicine to the store.

Code:

```
$categories = $pdo->query("SELECT * FROM categories ORDER BY name")
                  ->fetchAll(PDO::FETCH_ASSOC);

if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    $name        = clean($_POST['name']);
    $categoryId  = (int) $_POST['category_id'];
    $description = clean($_POST['description']);
    $price       = (float) $_POST['price'];
    $stock       = (int) $_POST['stock'];
    $threshold   = (int) $_POST['min_threshold'];
    $rx          = isset($_POST['requires_prescription']) ? 1 : 0;

    if ($name === '' || $price < 0 || $stock < 0) {
        $error = "Please fill in the form correctly.";
    } else {
        list($image, $imgError) = saveMedicineImage($_FILES['image'] ?? null, $name);
        $stmt = $pdo->prepare("INSERT INTO medicines (name, category_id, description,
                 price, stock, min_threshold, requires_prescription, image)
                 VALUES (?, ?, ?, ?, ?, ?, ?, ?)");
        $stmt->execute([$name, $categoryId, $description, $price,
                        $stock, $threshold, $rx, $image]);
        redirect('medicines.php');
    }
}
```

Explanation:

- **Category list:** all categories are read from the database to fill the Category drop-down.
- **Reading the form:** text is cleaned with clean(), numbers are cast to their types, and the checkbox becomes 1 or 0 for requires_prescription. The minimum threshold defaults to 10.
- **Validation:** the medicine is saved only when the name is given and the price and stock are not negative; otherwise an error message is shown.
- **Optional photo:** saveMedicineImage() checks the real file type and size before storing the photo in assets/uploads/medicines/.
- **Insert:** a prepared INSERT query adds the medicine and the admin returns to Manage Medicines.

admin/categories.php

Purpose: This page adds new categories and lists the existing ones with the number of medicines in each.

Code:

```
if ($_SERVER['REQUEST_METHOD'] === 'POST' && isset($_POST['add_category'])) {
    $name = clean($_POST['name']);
    if ($name === '') {
        $error = "Category name cannot be empty.";
    } else {
        $stmt = $pdo->prepare("SELECT id FROM categories WHERE name = ?");
        $stmt->execute([$name]);
        if ($stmt->fetch()) {
            $error = "A category with this name already exists.";
        } else {
            $stmt = $pdo->prepare("INSERT INTO categories (name) VALUES (?)");
            $stmt->execute([$name]);
            redirect('categories.php?added=1');
        }
    }
}

$categories = $pdo->query("
    SELECT c.*, COUNT(m.id) AS medicine_count
    FROM categories c LEFT JOIN medicines m ON m.category_id = c.id
    GROUP BY c.id ORDER BY c.name")->fetchAll(PDO::FETCH_ASSOC);
```

Explanation:

- **Add category:** an empty name is rejected, and a duplicate name is detected with a SELECT before the new category is inserted.
- **Medicine count:** a LEFT JOIN with GROUP BY counts the medicines in each category, so categories with no medicines still appear with 0.
- **Messages:** the added, deleted and blocked values in the URL show the matching message; blocked is shown when the admin tries to delete a category that still has medicines (checked in delete_category.php).
- **Actions:** each row has Edit (edit_category.php) and Delete links; Delete asks for confirmation first.

admin/customers.php

Purpose: This page lists all registered customers with their order count and total spending.

Code:

```
$customers = $pdo->query("
    SELECT u.*, COUNT(o.id) AS order_count,
           COALESCE(SUM(o.total_amount), 0) AS total_spent
    FROM users u
    LEFT JOIN orders o ON o.user_id = u.id
    WHERE u.role = 'customer'
    GROUP BY u.id
    ORDER BY u.created_at DESC
")->fetchAll(PDO::FETCH_ASSOC);

<td><a href="orders.php?user_id=<?php echo $cust['id']; ?>">
    <?php echo $cust['order_count']; ?></a></td>
<td>&#8377;<?php echo number_format($cust['total_spent'], 2); ?></td>
```

Explanation:

- **Customer list:** only users with the customer role are shown, newest first; the administrator account is never listed.
- **Order summary:** a LEFT JOIN with the orders table counts each customer's orders and adds their totals; COALESCE shows 0 for customers without orders.
- **Link to orders:** the order count opens admin/orders.php filtered to that customer.
- **Delete:** the Delete link (after confirmation) calls delete_customer.php, which deletes the account only if its role is customer, so the administrator account cannot be deleted from here; because of ON DELETE CASCADE, the customer's orders are removed too.

7.3 USER SIDE FILES:

register.php

Purpose: This page creates a new customer account.

Code:

```
if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    $name     = clean($_POST['full_name']);
    $email    = clean($_POST['email']);
    $password = $_POST['password'];
    $confirmPassword = $_POST['confirm_password'] ?? '';

    if ($name === '' || $email === '' || $password === '') {
        $error = "Please fill in all required fields.";
    } elseif ($password !== $confirmPassword) {
        $error = "Password and Confirm Password do not match.";
    } else {
        $stmt = $pdo->prepare("SELECT id FROM users WHERE email = ?");
        $stmt->execute([$email]);
        if ($stmt->fetch()) {
            $error = "An account with this email already exists.";
        } else {
            $hashed = password_hash($password, PASSWORD_DEFAULT);
            $stmt = $pdo->prepare("INSERT INTO users
                (full_name, email, password, phone, address, role)
                VALUES (?, ?, ?, ?, ?, 'customer')");
            $stmt->execute([$name, $email, $hashed, $phone, $address]);
            redirect('login.php?registered=1');
        }
    }
}
```

Explanation:

- **Validation:** required fields are checked and Password and Confirm Password must match. This is checked on the server as well as by assets/js/auth.js in the browser, so it cannot be skipped by turning JavaScript off.
- **Unique email:** a SELECT checks whether the email is already registered before a new account is created.
- **Password security:** the password is never stored as plain text; password_hash() stores a secure hash.

- **Role:** every account created here gets the customer role. After registration the user is sent to the Login page with a success message.

login.php

Purpose: This page logs in both customers and the administrator and sends each to the correct area.

Code:

```
$stmt = $pdo->prepare("SELECT * FROM users WHERE email = ?");
$stmt->execute([$email]);
$user = $stmt->fetch(PDO::FETCH_ASSOC);

if ($user && password_verify($password, $user['password'])) {
    $_SESSION['user_id'] = $user['id'];
    $_SESSION['name']    = $user['full_name'];
    $_SESSION['role']    = $user['role'];

    if ($user['role'] === 'admin') {
        redirect('admin/dashboard.php');
    } else {
        redirect('medicines.php');
    }
} else {
    $error = "Incorrect email or password.";
}
```

Explanation:

- **Finding the user:** the account is searched by email with a prepared statement.
- **Password check:** password_verify() compares the typed password with the stored hash.
- **Session:** on success the user id, name and role are stored in the session; these values are used by the login checks on every protected page.
- **Role-based redirect:** the administrator goes to the Admin Dashboard and a customer goes to the Medicines page.
- **Same error message:** a wrong email and a wrong password both show "Incorrect email or password.", so the page does not reveal which emails are registered.

medicines.php

Purpose: This is the main shopping page. It shows the medicines with search, category, price range and in-stock filters, sorting and pagination.

Code:

```
$perPage = 12;

$sortOptions = [
    'name_asc'   => ['Name: A to Z',       'm.name ASC'],
    'name_desc'  => ['Name: Z to A',       'm.name DESC'],
    'price_asc'  => ['Price: Low to High', 'm.price ASC, m.name ASC'],
    'price_desc' => ['Price: High to Low', 'm.price DESC, m.name ASC'],
];

if ($q !== '')          { $where[] = "m.name LIKE ?";      $params[] = '%' . $q . '%'; }
if ($categoryId > 0)    { $where[] = "m.category_id = ?";  $params[] = $categoryId; }
if ($activeMin !== null){ $where[] = "m.price >= ?";       $params[] = $activeMin; }
if ($activeMax !== null){ $where[] = "m.price <= ?";       $params[] = $activeMax; }
if ($inStock)           { $where[] = "m.stock > 0"; }

$sql = "SELECT m.*, c.name AS category_name
        FROM medicines m LEFT JOIN categories c ON c.id = m.category_id"
     . $whereSql
     . " ORDER BY " . $sortOptions[$sort][1]
     . " LIMIT " . (int)$perPage . " OFFSET " . (int)$offset;
```

Explanation:

- **Reading filters safely:** the search text, category, minimum and maximum price, in-stock option and sort order are read from the URL and validated; an unknown category or sort value falls back to the default.
- **Dynamic WHERE clause:** only the selected filters are added to the query, and every value is passed as a ? parameter of a prepared statement, preventing SQL injection. Special characters % and _ in the search text are escaped.
- **Sorting:** only the four allowed sort options can reach the ORDER BY part of the query.
- **Pagination:** a COUNT(*) query finds the number of matching medicines; LIMIT and OFFSET then show 12 medicines per page.
- **Display:** each card shows the photo, category, name, description, price, stock and an Rx Required tag for prescription-only medicines, with a quantity box and an Add to Cart form that carries the CSRF token. Active filters are shown as removable chips, and the price slider uses assets/js/medicines.js.

add_to_cart.php

Purpose: This script adds the selected medicine to the customer's cart, which is stored in the session.

Code:

```
requireCustomerLogin();

if ($_SERVER['REQUEST_METHOD'] === 'POST' && verifyCsrf($_POST['csrf_token'] ?? '')) {
    $medicineId = (int) ($_POST['medicine_id'] ?? 0);
    $qty = max(1, (int) ($_POST['qty'] ?? 1));

    $stmt = $pdo->prepare("SELECT * FROM medicines WHERE id = ?");
    $stmt->execute([$medicineId]);
    $med = $stmt->fetch(PDO::FETCH_ASSOC);

    if ($med && $med['stock'] > 0) {
        $already = isset($_SESSION['cart'][$medicineId])
                 ? (int) $_SESSION['cart'][$medicineId]['qty'] : 0;
        $newQty  = min($already + $qty, (int) $med['stock']);
        ...   // add new line or increase the quantity
    }
}
redirect('medicines.php?' . http_build_query($back) . '#shop-results');
```

Explanation:

- **Checks:** only a logged-in customer can add items, and the request must be a POST with a valid CSRF token.
- **Medicine lookup:** the medicine is read from the database, and it is added only if it exists and is in stock.
- **Stock limit:** the quantity already in the cart plus the new quantity can never be more than the available stock.
- **Cart structure:** the cart is $_SESSION['cart'], keyed by medicine id, holding the name, price, quantity, prescription flag and photo.
- **Return:** the customer is sent back to the same filtered, sorted page of the medicines list they were viewing.

upload_prescription.php

Purpose: This script uploads the prescription required for prescription-only medicines and stores it safely on the server.

Code:

```
$cartInfo = getCartDetails($pdo);
if (!$cartInfo['needs_rx']) { redirect('cart.php'); }   // no Rx medicine in cart

$allowed = [
    'jpg'  => ['image/jpeg'], 'jpeg' => ['image/jpeg'],
    'png'  => ['image/png'],  'pdf'  => ['application/pdf'],
];
$ext = strtolower(pathinfo($file['name'], PATHINFO_EXTENSION));

if (!isset($allowed[$ext])) {
    $error = "Only JPG, PNG, or PDF files are allowed.";
} elseif ($file['size'] > 5 * 1024 * 1024) {
    $error = "File is too large (max 5MB).";
} else {
    $finfo = new finfo(FILEINFO_MIME_TYPE);
    $mime  = $finfo->file($file['tmp_name']);
    if (!in_array($mime, $allowed[$ext], true)) {
        $error = "The file content does not match a JPG, PNG or PDF.";
    } else {
        $newName = 'rx_' . $_SESSION['user_id'] . '_' . time() . '_'
                 . bin2hex(random_bytes(4)) . '.' . $ext;
        move_uploaded_file($file['tmp_name'], __DIR__ . '/assets/uploads/' . $newName);
        $_SESSION['prescription_path'] = 'assets/uploads/' . $newName;
    }
}
```

Explanation:

- **When it is needed:** the cart is checked from the database; if it has no prescription-only medicine, the upload is not needed and the customer is sent back.
- **File type check:** only the jpg, jpeg, png and pdf extensions are allowed, and the real content type is checked with finfo, so a renamed text or script file is rejected.
- **Size limit:** files larger than 5 MB are rejected.
- **Safe file name:** the file is saved with a new server-generated name (rx_<user id>_<time>_<random>), so nothing from the original file name is used.
- **Linking to the order:** the saved path is kept in the session and attached to the next order at checkout. The upload box (includes/rx_upload_box.php) is shown on the cart and checkout pages, and the result is shown there with a flash message.

checkout.php

Purpose: This page places the order. It saves the order and its items and reduces the stock inside one database transaction.

Code:

```
$cartInfo = getCartDetails($pdo);
if ($needsPrescription && $prescriptionPath === null) {
    $error = "Please upload your prescription before placing this order.";
} else {
    $pdo->beginTransaction();

    // 1. lock the medicine rows and re-check stock and price
    $lock = $pdo->prepare("SELECT id, name, price, stock, requires_prescription
                           FROM medicines WHERE id IN ($in) FOR UPDATE");

    // 2. create the order
    $stmt = $pdo->prepare("INSERT INTO orders
              (user_id, total_amount, prescription_path, status)
              VALUES (?, ?, ?, 'pending')");
    $orderId = $pdo->lastInsertId();

    // 3. save each item and reduce stock
    $itemStmt  = $pdo->prepare("INSERT INTO order_items
              (order_id, medicine_id, quantity, price) VALUES (?, ?, ?, ?)");
    $stockStmt = $pdo->prepare("UPDATE medicines SET stock = stock - ?
                                WHERE id = ? AND stock >= ?");
    $pdo->commit();

    // 4. clear the cart and prescription
    unset($_SESSION['cart'], $_SESSION['prescription_path']);
    redirect('order_success.php?order_id=' . $orderId);
}
// on any error: $pdo->rollBack();
```

Explanation:

- **Server-side checks:** the CSRF token is verified, the cart is rebuilt from database prices, and the order is refused if a prescription is needed but not uploaded. If stock changed since the cart was built, the customer is asked to review the order first.
- **Transaction:** beginTransaction(), commit() and rollBack() make sure the order, its items and the stock changes are saved together or not at all.
- **Row locking:** SELECT ... FOR UPDATE locks the medicine rows so two customers cannot buy the last units at the same time; stock and price are checked again inside the transaction.
- **Saving the order:** a row is added to orders with status pending (the prescription path is attached only when needed), and one row per medicine is added to order_items with the database price.
- **Stock reduction:** the UPDATE uses "stock >= ?" so stock can never go below zero.
- **Finish:** the cart and prescription are cleared from the session and the customer is redirected to the Order Placed page. Payment is Cash on Delivery.

my_orders.php

Purpose: This page shows the logged-in customer their own orders and lets them cancel an order that has not yet shipped.

Code:

```
$stmt = $pdo->prepare("SELECT * FROM orders WHERE user_id = ?
                       ORDER BY created_at DESC, id DESC");
$stmt->execute([$_SESSION['user_id']]);
$orders = $stmt->fetchAll(PDO::FETCH_ASSOC);

$stmt = $pdo->prepare(
    "SELECT oi.order_id, oi.quantity, oi.price, m.name
     FROM order_items oi
     JOIN medicines m ON m.id = oi.medicine_id
     JOIN orders o    ON o.id = oi.order_id
     WHERE o.user_id = ? ORDER BY oi.id");
$stmt->execute([$_SESSION['user_id']]);
foreach ($stmt->fetchAll(PDO::FETCH_ASSOC) as $row) {
    $itemsByOrder[$row['order_id']][] = $row;
}

<?php if (isOrderCancellable($order['status'])): ?>
    <form method="post" action="cancel_order.php"> ... Cancel Order ... </form>
<?php endif; ?>
```

Explanation:

- **Own orders only:** both queries use the user id from the session, so a customer can never see another customer's orders.
- **Two queries instead of many:** all order items are fetched in one query and grouped by order id in PHP, instead of running a separate query for every order.
- **Order card:** each order shows its number, date, status badge with an icon, the items with price, quantity and subtotal, whether a prescription is attached, and the total.
- **Cancel Order:** the button is shown only for pending or processing orders (isOrderCancellable()); the request goes to cancel_order.php with a CSRF token after a confirmation message.

7.4 DATABASE:

database.sql

Purpose: This script creates the medipharma_hub database with its five tables and fills it with the starting data: the administrator account, 12 categories and 127 medicines.

Code:

```
CREATE TABLE users (
    id INT AUTO_INCREMENT PRIMARY KEY,
    full_name VARCHAR(100) NOT NULL,
    email VARCHAR(150) NOT NULL UNIQUE,
    password VARCHAR(255) NOT NULL,
    phone VARCHAR(20), address VARCHAR(255),
    role ENUM('customer','admin') NOT NULL DEFAULT 'customer',
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
CREATE TABLE medicines (
```

```
    id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(150) NOT NULL,
    category_id INT,
    price DECIMAL(10,2) NOT NULL DEFAULT 0,
    stock INT NOT NULL DEFAULT 0,
    min_threshold INT NOT NULL DEFAULT 10,
    requires_prescription TINYINT(1) NOT NULL DEFAULT 0,
    image VARCHAR(255) DEFAULT NULL,
    FOREIGN KEY (category_id) REFERENCES categories(id) ON DELETE SET NULL
);
CREATE TABLE orders (
    id INT AUTO_INCREMENT PRIMARY KEY,
    user_id INT NOT NULL,
    total_amount DECIMAL(10,2) NOT NULL DEFAULT 0,
    prescription_path VARCHAR(255) DEFAULT NULL,
    status ENUM('pending','processing','shipped','delivered','cancelled') DEFAULT 'pending',
    FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE
);
CREATE TABLE order_items (
    id INT AUTO_INCREMENT PRIMARY KEY,
    order_id INT NOT NULL, medicine_id INT NOT NULL,
    quantity INT NOT NULL, price DECIMAL(10,2) NOT NULL,
    FOREIGN KEY (order_id) REFERENCES orders(id) ON DELETE CASCADE,
    FOREIGN KEY (medicine_id) REFERENCES medicines(id) ON DELETE CASCADE
);
```

Explanation:

- **users:** stores customers and the administrator; the email is UNIQUE, the password is stored as a hash and the role column separates customer and admin.
- **categories:** stores the 12 medicine categories (Nutritional Supplements, Ayurvedic Products, Diabetes Care, Heart Care and so on).
- **medicines:** stores each product with price, stock, minimum stock threshold (used for the low-stock alert), the requires_prescription flag and the photo name. If a category is deleted, ON DELETE SET NULL keeps the medicine.
- **orders:** stores each order with the customer, total amount, prescription path and status (pending, processing, shipped, delivered or cancelled).
- **order_items:** stores the medicines in every order with the quantity and the price at the time of ordering, so later price changes do not affect old orders.
- **Relationships:** foreign keys with ON DELETE CASCADE remove a customer's orders and their items automatically when the customer is deleted.
- **Seed data:** one administrator account (admin@medipharma.com, with a hashed password), 12 categories and 127 medicines with photos are inserted so the system is ready to use.

8. LIMITATIONS AND FUTURE ENHANCEMENTS

8.1 LIMITATIONS:

- **Cash on Delivery only:** there is no online payment gateway.
- **No e-mail or SMS notification:** the low-stock alert is shown only on the admin dashboard and customers get no automatic message about their order.
- **No invoice:** an order total is shown on screen but no invoice can be downloaded.
- **Cart is stored in the session:** the cart and the uploaded prescription path are lost when the session ends; they are not saved in the database.
- **Manual prescription checking:** the administrator opens the uploaded file from the Orders page; there is no approve / reject step for prescriptions.
- **Single seeded administrator:** only one admin account is created by database.sql and there is no page to create more.
- **No password recovery or password change:** the profile page edits name, email, phone and address only.
- **Admin actions:** deleting medicines, categories and customers uses simple links, and the order-status form of the admin panel does not use a CSRF token or restore stock when the admin sets an order to "cancelled" (stock is restored only when the customer cancels).
- **Deleting data:** deleting a customer also deletes the customer's orders, and deleting a medicine removes it from old order items, because of the CASCADE rules.
- **Basic reports:** the dashboard shows simple counts only.
- **Local setup:** the database port in config/db.php is set for the local MySQL (3306) and must be changed if the server differs.

8.2 FUTURE ENHANCEMENTS:

- Online payment integration (UPI, cards, net banking).
- E-mail and SMS notifications for orders and low stock.
- Invoice download for every order.
- Saving the cart in the database so that it survives logout.
- A prescription verification workflow where the pharmacist approves or rejects a prescription.
- Multiple admin roles (for example admin and pharmacist) and creation of admin accounts.
- Forgot password / password change with e-mail verification.

- CSRF protection for all admin forms and confirmation by POST for delete actions.
- Medicine expiry date, batch and supplier management.
- Sales reports and analytics (daily, monthly, yearly).

9. CONCLUSION

The project **MediPharma Hub – Online Pharmacy and Medicine Order Portal** has been completed successfully. The system provides a simple and organized way for customers to search and filter medicines by name or category, add them to a cart, upload a doctor's prescription when it is required, and place an order with Cash on Delivery, without visiting the pharmacy. Customers can also update their profile, view the order history and the current status of every order on the My Orders page, and cancel an order that has not yet been shipped.

For the pharmacy, the system replaces manual registers with a single admin panel. The administrator can manage medicines, categories, customers and orders, view the uploaded prescriptions, and update the order status. The low-stock alert on the dashboard shows the medicines that need to be refilled, which helps to avoid running out of stock. The system uses role-based access, hashed passwords and prepared statements, which helps to keep the data of the customers and the pharmacy secure.

During the development of this project, I learned how to design a database using normalization, build a dynamic website using PHP and MySQL, handle sessions, file uploads and transactions, and prepare test cases for each module. The project has some limitations, such as the absence of an online payment option and email or SMS notifications, which are explained in the previous chapter. The system is built in a modular way, so these features can be added in the future, and I believe it forms a strong base for a complete online pharmacy solution.

10. REFRENCERS AND BIBLIOGRAPHY

All the references listed below were used for learning, developing and designing this project and for preparing this documentation.

Books:

- PHP & MySQL Web Development – Luke Welling and Laura Thomson
- Learning PHP, MySQL & JavaScript – Robin Nixon

Tools and Software:

- XAMPP (Apache, MySQL, PHP)
- phpMyAdmin (Database Management)
- Visual Studio Code (Code Editor)
- Font Awesome (Icons)
- Google Fonts (Fonts)

Websites:

- www.php.net (PHP Manual)
- dev.mysql.com (MySQL Documentation)
- www.w3schools.com
- www.stackoverflow.com
- chatgpt.com